

MASTER'S THESIS COMPUTER SCIENCE



RADBOD UNIVERSITY NIJMEGEN

Infinite Control Improvisation

by

Jasper Laumen

27th July 2026

First assessor: Dr Jurriaan Rot

Second assessor: Dr Sebastian Junges

Abstract

Many applications require behaviour that is unpredictable yet still satisfies strict requirements, from automated testing to route planning. Control improvisation (CI) provides a framework for specifying such requirements through hard constraints, which must always be satisfied, and soft constraints, which must be satisfied with some minimum probability, and constructs a random generator that meets these requirements. However, classical CI has several limitations: it requires a fixed upper bound on improvisation length, assigns uniform lower and upper probability bounds to all outputs rather than individual bounds, and supports only limited forms of soft constraints. We introduce weighted control improvisation (WCI), which uses weighted finite automata to generalise classical CI and overcome these limitations. We investigate the verification and synthesis problems for WCI and show that, although they are undecidable in full generality, practically relevant instances admit verification and synthesis algorithms.

Contents

1	Introduction • 1
1.1	Overview • 3
1.2	Contributions and Outline • 4
2	Preliminaries • 5
2.1	Weighted Languages • 5
2.2	Weighted Finite Automata • 8
2.3	Operations on WFAs • 11
2.4	Minimisation • 12
3	Control Improvisation • 15
3.1	Weighted Control Improvisation • 15
3.2	Classical Control Improvisation • 19
3.3	A Randomised Park-Route Example • 22
4	Weighted Finite Automata • 25
4.1	Computing the Total Weight • 25
4.2	Sampling from Non-Stochastic WFAs • 34
4.3	The Containment Problem • 37
4.4	Approximating the Containment Problem • 41
5	Solving the WCI Problem • 45
5.1	Verification • 45
5.2	Synthesis • 47
5.2.1	General Synthesis • 47
5.2.2	Eliminating the Lower Bound • 49
5.2.3	Synthesis without Soft Constraints • 51
5.2.4	A Greedy Approach • 53
5.2.5	Finitely Insufficient Soft Constraints • 55
5.3	Synthesis with Deterministic Soft Constraints • 61
6	Related Work • 68
6.1	Weighted automata and rational series • 68
6.2	Probabilistic automata and stochastic languages • 69
6.3	Control improvisation • 69

6.4	Extensions of control improvisation •	70
6.5	Positioning of WCI •	70
7	Conclusion •	71
7.1	Future work •	72

1 Introduction

Many systems are required to generate behaviour that is both correct and unpredictable. This situation appears wherever randomisation is a design objective rather than an inconvenience: in randomised testing, where repeatedly exercising the same execution path defeats the purpose; in music generation, where a melody should vary between performances while remaining harmonically consistent with its accompaniment [6]; in surveillance and patrol scheduling, where a predictable strategy can be anticipated and evaded [23]; and in routing and scheduling, where some admissible outputs are more valuable than others. A deterministic generator is too rigid, producing exploitable regularities. A purely random generator is too unconstrained, potentially violating feasibility requirements or wasting probability mass on outputs that serve no purpose. The goal is a generator that is simultaneously feasible, sufficiently unpredictable, and effective in expectation.

A concrete instance of this problem arises in fare inspection on public transit. Consider a train company that employs a conductor, Bob, to check tickets on its network. If Bob always patrols the same route, passengers quickly learn which routes are safe, and fare evasion simply moves there. If his route is chosen uniformly at random, a busy route might go unchecked for a week while a quiet one is inspected far too often. What the company actually wants lies in between: a random schedule that is unpredictable enough to deter evasion everywhere, yet structured enough to ensure that, on average, enough tickets are checked and that busier routes are covered more often than marginal ones. Transit agencies face exactly this trade-off in practice and have turned to algorithmic, game-theoretic patrol scheduling to address it [25, 23].

In general, we could desire a random generator satisfying three kinds of requirements simultaneously. A *hard* requirement specifies which behaviours may ever be produced: Bob's route must follow the rail network. *Randomness* requirements bound the probability of each individual output, ensuring the generator is neither too predictable nor too sparse. A *soft* requirement asks that the generator perform well on average: Bob should check enough tickets per shift to make fare evasion unattractive.

The framework of control improvisation (CI), introduced by Fremont, Donzé, Seshia, and Wessel [8], formalises this setting. A CI solution generates only words from a given regular (even finite) language, ensures every admissible word of bounded length receives probability in a fixed interval $[\lambda, \rho]$, and satisfies a Boolean soft language constraint with high probability.

For Bob’s network, classical CI [8] can guarantee valid routes, bound the probability of each route, and ensure some route property holds often. However, it comes with some limitations:

- We cannot make a distinction between a busy route carrying 200 passengers and a quiet one carrying fifty, since every admissible word receives the same bounds $[\lambda, \rho]$.
- It cannot express a quantitative objective such as “check at least 1 000 passengers on average per shift”, since the soft constraint is Boolean.
- Because the candidate set must be finite, it requires an artificial upper bound on word length, even though Bob’s shifts have no natural maximum.

Our main goal is to develop a framework, which generalises CI, and also addresses the above limitations. This thesis introduces *weighted control improvisation* (WCI), which addresses these limitations by replacing the regular-language specifications of classical CI with *weighted finite automata* (WFAs). A WFA assigns a numerical value to each word rather than a simple yes/no, making it possible to express word-dependent probability bounds and genuine quantitative soft objectives within the same linear algebraic framework. WCI is a strict generalisation of classical CI: every classical instance can be encoded as a weighted one, but the converse does not hold. We return to Bob’s network throughout the thesis as a running example.

1.1 Overview

A WCI instance consists of three WFAs (formally introduced in Chapter 2). A *lower-bound automaton* specifies, for each word, the minimum probability it must receive. An *upper-bound automaton* specifies the maximum. A *soft-constraint automaton* assigns a score to each word, with the requirement that the expected score of the generated distribution exceeds a given threshold.

For Bob’s network, the upper-bound automaton encodes which routes are operationally valid, the lower-bound automaton ensures every feasible route receives some non-zero probability, and the same pair of automata can assign a busy route a higher minimum probability than a marginal one, something a single global interval could never express. The soft-constraint automaton counts the passengers checked along a route, turning “check enough tickets on average” into the precise condition that the expected score exceeds a given threshold. A solution to the WCI problem is then a stochastic WFA that respects all three requirements simultaneously.

WFAs are a well-studied model with strong algorithmic properties. In particular, we study questions about the total weight of a WFA. These questions are essential for a probabilistic interpretation of WFAs, so we develop efficient computation methods (Chapter 4).

Given a WCI instance, the *synthesis problem* asks whether a valid solution exists and, if so, how to construct one. The *verification problem* asks whether a given stochastic WFA is a valid solution. Both problems are undecidable in full generality, a consequence of corresponding undecidability results for probabilistic automata [3, 4]. Nevertheless, tractable fragments exist: for practical instances of the WCI problem, we are still able to construct and verify solutions.

Control improvisation was introduced by Fremont, Donzé, Seshia, and Wessel [8], originally motivated by music generation, where the goal is to produce random variants of a melody that remain harmonically and structurally consistent with its accompaniment [6]. The same pattern appears in surveillance, home automation, and randomised supervisory control, where a deterministic strategy is strategically transparent and therefore easier to circumvent.

More formally, a classical CI instance is a tuple $(H, S, \epsilon, \lambda, \rho, n)$, where $H, S \subseteq \Sigma^*$ are regular languages, $0 \leq \epsilon \leq 1$, $0 \leq \lambda \leq \rho \leq 1$, and $n \in \mathbb{N}$ bounds the word length. A probabilistic generator is a solution if every generated word lies in H and has length at most n , every such word receives probability between λ and ρ , and the soft language S is satisfied with probability at least $1 - \epsilon$. The original paper gave a polynomial-time improvisation scheme for finite-memory specifications of this form, while showing that more expressive variants are #P-hard

or undecidable, a complexity typical of synthesis problems.

WCI strictly generalises classical CI. The classical parameters λ , ρ , and ϵ can be encoded directly as WFAs, so every classical CI instance becomes a special case of a weighted one (Chapter 3). The weighted framework goes beyond the classical one in three ways: it allows word-dependent lower and upper probability bounds rather than a single global interval; it supports improvisations of arbitrary length rather than requiring a fixed maximum; and it permits genuine quantitative soft objectives such as expected scores, which classical CI cannot express.

Classical CI has also been extended along other axes. Akkaya et al. considered CI with probabilistic temporal specifications [1], Fremont and Seshia extended it to reactive settings [9], and Vazquez-Chanlatte et al. formalised randomness requirements via causal entropy in stochastic games [24]. We discuss these extensions and their relation to WCI in Chapter 6.

1.2 Contributions and Outline

This thesis makes the following main contributions:

- Introduce the weighted control improvisation framework as a generalisation of control improvisation.
- Study examples of weighted control improvisation problems.
- Describe algorithms for the verification and synthesis problems.
- Develop an approximation algorithm for the containment problem.

The thesis is structured as follows. Chapter 2 introduces the necessary preliminaries: weighted languages, weighted finite automata, operations on WFAs, and minimisation via forward and backward spaces. Chapter 3 defines WCI formally, compares it to classical CI, and develops toy examples. Chapter 4 studies the algorithmic properties of weighted finite automata: computing total weights, sampling, the containment problem, and its approximation. Chapter 5 then addresses the two main computational problems of weighted control improvisation, verification and synthesis, including a synthesis procedure for a restricted class of soft-constraint automata. Chapter 6 situates the work in the broader literature on weighted automata, probabilistic automata, and control improvisation.

2 Preliminaries

In this chapter, we introduce weighted languages and weighted finite automata (WFAs), which extend the classical notions of languages and finite automata by associating numerical weights with words and transitions. Weighted automata provide a natural framework for modelling quantitative properties of languages, such as probabilities, costs, or frequencies, while retaining many of the algorithmic advantages of finite-state models.

We begin by defining weighted languages and studying when their total weight is well-defined. We then introduce WFAs and show how they induce weighted languages through a matrix-based semantics. This linear algebraic perspective allows us to reason about weighted automata using tools from matrix theory and vector spaces. After establishing the basic definitions, we present several operations on WFAs and show how they correspond to natural operations on weighted languages. Finally, we discuss minimisation, where linear algebraic techniques can be used to compute canonical representations of weighted automata with the smallest possible number of states.

The concepts developed in this chapter form the foundation for the weighted control improvisation framework studied later in this thesis.

2.1 Weighted Languages

Recall that a language can be viewed as a function $\Sigma^* \rightarrow \{0, 1\}$ that assigns to each word either the value 1 (accepted) or 0 (rejected). Weighted languages generalise this notion by allowing arbitrary real-valued weights. Rather than simply determining whether a word belongs to a language, a weighted language assigns a numerical value to each word. Depending on the application, these values may represent probabilities, costs, rewards, frequencies, or other quantitative measures.

Throughout this thesis, we fix a finite alphabet Σ , unless we specifically mention otherwise.

► **Definition 2.1** (Weighted language). A *weighted language* is a function

$$L : \Sigma^* \rightarrow \mathbb{R}.$$

For a word $w \in \Sigma^*$, the value $L(w)$ is called the *weight* of w . We say that L is *non-negative* if

$$L(w) \geq 0$$

for all $w \in \Sigma^*$.

A particularly important class of weighted languages are those that can be interpreted as probability distributions over words. To make this precise, we first need a notion of the total weight of a language. Given a weighted language L , one would naturally like to define its total weight as

$$\sum_{w \in \Sigma^*} L(w).$$

Since Σ^* is countably infinite, this sum must be interpreted as a limit. A natural definition is

$$\lim_{n \rightarrow \infty} \sum_{k=0}^n \sum_{w \in \Sigma^k} L(w).$$

For general weighted languages, however, the existence of this limit alone is not sufficient. Consider the weighted language over the alphabet $\{a, b\}$ defined by

$$L(a^n) = n, \quad L(b^n) = -n,$$

and $L(w) = 0$ for all other words. The positive and negative contributions cancel exactly, so the above limit evaluates to 0, despite the fact that words with arbitrarily large weight exist. Since such behaviour is undesirable for the applications considered in this thesis, we additionally require that the weights of individual words vanish as the word length increases.

► **Definition 2.2** (Weight of a language). Let L be a weighted language. We say that L has *finite weight* if

$$\lim_{n \rightarrow \infty} \sum_{k=0}^n \sum_{w \in \Sigma^k} L(w)$$

exists and is finite, and if

$$\lim_{k \rightarrow \infty} \max_{w \in \Sigma^k} |L(w)| = 0.$$

The value of the first limit is called the *weight* of L and is denoted by $\text{weight}(L)$.

A weighted language is called *stochastic* if it is non-negative and has weight 1.

The second condition rules out situations in which large positive and negative weights cancel each other in the limit. For non-negative weighted languages, however, this condition follows automatically from the existence of a finite total weight. Since weighted control improvisation will ultimately be concerned with probability distributions, non-negative weighted languages will be of particular interest.

► **Proposition 2.3.** Let L be a non-negative weighted language. If

$$\lim_{n \rightarrow \infty} \sum_{k=0}^n \sum_{w \in \Sigma^k} L(w)$$

exists and is finite, then

$$\lim_{k \rightarrow \infty} \max_{w \in \Sigma^k} L(w) = 0.$$

Consequently, L has finite weight.

Proof. Assume for contradiction that

$$\lim_{k \rightarrow \infty} \max_{w \in \Sigma^k} L(w) \neq 0.$$

Then there exists a constant $c > 0$ and infinitely many integers k such that

$$\max_{w \in \Sigma^k} L(w) \geq c.$$

For each such k , choose a word $w_k \in \Sigma^k$ satisfying

$$L(w_k) \geq c.$$

Since L is non-negative,

$$\sum_{w \in \Sigma^k} L(w) \geq L(w_k) \geq c$$

for infinitely many values of k .

It follows that the partial sums

$$\sum_{j=0}^n \sum_{w \in \Sigma^j} L(w)$$

increase by at least c infinitely often. Therefore, they cannot converge to a finite limit, contradicting the assumption.

Hence,

$$\lim_{k \rightarrow \infty} \max_{w \in \Sigma^k} L(w) = 0,$$

which is what we were tasked to prove. ◀

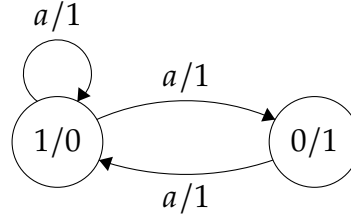


Figure 2.1: A WFA computing the Fibonacci sequence

2.2 Weighted Finite Automata

Similarly to how we extended the notion of a language to a weighted language, we will extend the notion of a deterministic finite automaton (DFA) to that of a *weighted finite automaton* (WFA), by equipping each transition with a weight, and additionally equipping each state with an initial and final weight (which replace the initial and final states).

Just as DFAs induce a language, WFAs induce a weighted language. The weight of a path

$$q_0 \xrightarrow{\sigma_1} q_1 \xrightarrow{\sigma_2} \dots \xrightarrow{\sigma_n} q_n$$

is the product of the initial weight of q_0 , the weights of all transitions along the path, and the final weight of q_n . The weight assigned to a word w is the sum of the weights of all paths labelled by w .

Figure 2.1 shows an example of a WFA. The two values shown in the states are the initial and final weights respectively. Each transition shows the input symbol that corresponds to that transition, and its weight. Here, there are two different paths that accept the word “aaa”, namely $q_0 \rightarrow q_0 \rightarrow q_0 \rightarrow q_1$ and $q_0 \rightarrow q_1 \rightarrow q_0 \rightarrow q_1$, both of which have a weight of 1. Thus, the weight of the word “aaa” is 2. In general, this WFA generates the Fibonacci sequence, illustrating the expressive power of even very small WFAs.

The weight of a word can be computed efficiently using matrix multiplication, where each transition matrix records the weights of transitions labelled by a fixed symbol. This linear algebraic viewpoint is central to the theory of weighted automata and will be useful throughout this work.

► **Definition 2.4** (Weighted finite automaton). A *weighted finite automaton* (WFA) is a tuple $\mathcal{W} = (\Sigma, n, \alpha, \mu, \beta)$, where

- Σ is a finite alphabet;
- $n \in \mathbb{N}$ is the number of states;
- $\alpha \in \mathbb{R}^{1 \times n}$ is the *initial vector*;

- $\mu : \Sigma \rightarrow \mathbb{R}^{n \times n}$ is a map of *transition matrices*;
- $\beta \in \mathbb{R}^{n \times 1}$ is the *final vector*.

Comparing this definition to our state-based approach, the values α_i and β_i correspond respectively to the initial weight and the final weight of state i , and $\mu(\sigma)_{ij}$ corresponds to the weight of the transition $q_i \xrightarrow{\sigma} q_j$. We also say that the number of states n is the *size* of $\mathcal{W}$.

Using this definition of WFAs, we are able to compute weights using matrix multiplication.

► **Definition 2.5.** We recursively extend μ to a map $\Sigma^* \rightarrow \mathbb{R}^{n \times n}$ by defining $\mu(\varepsilon) = I_n$ (the identity matrix of size n) and $\mu(w\sigma) = \mu(w)\mu(\sigma)$ for all $\sigma \in \Sigma$ and $w \in \Sigma^*$. The *semantics* of $\mathcal{W}$ is a weighted language $\mathcal{L}(\mathcal{W}) : \Sigma^* \rightarrow \mathbb{R}$, defined as

$$\mathcal{L}(\mathcal{W})(w) = \alpha\mu(w)\beta.$$

We misuse the notation and write $\text{weight}(\mathcal{W}) = \text{weight}(\mathcal{L}(\mathcal{W}))$. We say $\mathcal{W}$ is *stochastic* if

- $\sum_{i=1}^n \alpha_i = 1$, and
- $\sum_{\sigma \in \Sigma} \sum_{j=1}^n \mu(\sigma)_{ij} + \beta_i = 1$ for all $i \in \{1, \dots, n\}$.

In this case, it also holds that $\text{weight}(\mathcal{W}) = 1$ (so the language forms a probability distribution). In other words, the initial weights form a probability distribution, and the outgoing weights together with the final weight of a state forms a probability distribution for each state.

Now, we will formulate three examples of WFAs. The first example is the WFA of Figure 2.1, but now in our formal definition. The second example is an example of a stochastic automaton that can be used as a sampler of words over the language $\mathcal{L}((a+b)^*)$. The third example shows a counting WFA that allows for weights to be linear combinations of the number of occurrences of a letter.

► **Example 2.6.** Define the WFA $\mathcal{W} = (\{a\}, 2, \alpha, \mu, \beta)$, where

$$\alpha = \begin{pmatrix} 1 & 0 \end{pmatrix} \quad \mu(a) = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \quad \beta = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

The weight of the word “aaa” is given by $\alpha \cdot \mu(a) \cdot \mu(a) \cdot \mu(a) \cdot \beta = 2$.

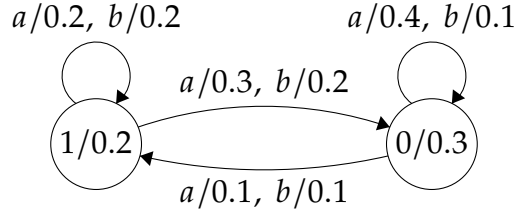
► **Example 2.7.** Consider the WFA $\mathcal{W} = (\{a, b\}, 2, \alpha, \mu, \beta)$ with

$$\alpha = \begin{pmatrix} 1 & 0 \end{pmatrix} \quad \beta = \begin{pmatrix} 0.2 \\ 0.3 \end{pmatrix},$$

and

$$\mu(a) = \begin{pmatrix} 0.2 & 0.3 \\ 0.1 & 0.4 \end{pmatrix} \quad \mu(b) = \begin{pmatrix} 0.1 & 0.2 \\ 0.1 & 0.1 \end{pmatrix}.$$

For convenience, we also illustrate $\mathcal{W}$ graphically:



We may view $\mathcal{W}$ as a random generator of words. First, an initial state is sampled according to α . Then, in the current state i , the automaton either stops with probability β_i , or chooses a symbol $\sigma \in \{a, b\}$ together with a next state j according to the outgoing probabilities $\mu(\sigma)_{ij}$. After moving to the new state j , the same procedure is repeated from there.

Since $\alpha = (1, 0)$, the process starts in state 1 with probability 1. For instance, one possible run is

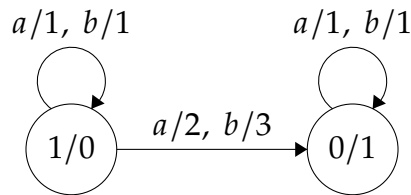
$$1 \xrightarrow{a} 2 \xrightarrow{b} 1 \xrightarrow{\text{stop}} .$$

This should be read as follows: starting in state 1, the automaton first chooses the transition labelled a to state 2. From state 2, it then chooses a transition from the outgoing distribution of that state, here the transition labelled b to state 1. Finally, in state 1, it terminates. The probability of this run is

$$\mu(a)_{1,2} \mu(b)_{2,1} \beta_1 = 0.3 \cdot 0.1 \cdot 0.2 = 0.006.$$

The probability $\mathcal{L}(\mathcal{W})(w)$ of a word w is obtained by summing the probabilities of all runs that generate w .

► **Example 2.8** (Counting WFA). Consider the following WFA.



In this WFA, the weight of a word w is given by $2|w|_a + 3|w|_b$. In fact, any such linear combination can be represented by such a WFA with two states.

With the basic definition out of the way, some natural questions arise. Can we combine WFAs? Can we minimise them? Can we compare them? We will tackle these problems in the following sections, and the resulting theory will be used when studying control improvisation in the setting of weighted automata.

2.3 Operations on WFAs

There are several binary operations for DFAs, such as the union, difference, and product. These operations correspond to operations on WFAs, and we will define them such that their languages behave as expected. The class of weighted languages recognised by WFAs is closed under several natural operations. In particular, we can construct WFAs whose semantics correspond to the pointwise sum, product, and scalar multiplication of weighted languages.

- **Definition 2.9** (Sum of WFAs). Let $\mathcal{W}_1 = (\Sigma, n, \alpha_1, \mu_1, \beta_1)$ and $\mathcal{W}_2 = (\Sigma, m, \alpha_2, \mu_2, \beta_2)$ be WFAs. The *sum* of $\mathcal{W}_1$ and $\mathcal{W}_2$, denoted $\mathcal{W}_1 + \mathcal{W}_2$, is given by the WFA $(\Sigma, n + m, \alpha, \mu, \beta)$, where α , μ and β are block matrices given by

$$\alpha = \begin{pmatrix} \alpha_1 & \alpha_2 \end{pmatrix} \quad \mu(\sigma) = \begin{pmatrix} \mu_1(\sigma) & 0 \\ 0 & \mu_2(\sigma) \end{pmatrix} \quad \beta = \begin{pmatrix} \beta_1 \\ \beta_2 \end{pmatrix}$$

- **Proposition 2.10.** Let $\mathcal{W}_1, \mathcal{W}_2$ be WFAs. Then, we have $\mathcal{L}(\mathcal{W}_1 + \mathcal{W}_2) = \mathcal{L}(\mathcal{W}_1) + \mathcal{L}(\mathcal{W}_2)$.

The following construction corresponds to pointwise multiplication of weighted languages and is often called the Hadamard product in the literature.

- **Definition 2.11** (Product of WFAs). Let $\mathcal{W}_1 = (\Sigma, n, \alpha_1, \mu_1, \beta_1)$ and $\mathcal{W}_2 = (\Sigma, m, \alpha_2, \mu_2, \beta_2)$ be WFAs. The *product* of $\mathcal{W}_1$ and $\mathcal{W}_2$, denoted $\mathcal{W}_1 \odot \mathcal{W}_2$, is given by the WFA $(\Sigma, n \cdot m, \alpha, \mu, \beta)$, where

$$\alpha = \alpha_1 \otimes \alpha_2 \quad \mu(\sigma) = \mu_1(\sigma) \otimes \mu_2(\sigma) \quad \beta = \beta_1 \otimes \beta_2.$$

Intuitively, the product construction runs both automata in parallel, multiplying the weights accumulated by each automaton.

- **Proposition 2.12.** Let $\mathcal{W}_1, \mathcal{W}_2$ be WFAs. Then, we have $\mathcal{L}(\mathcal{W}_1 \odot \mathcal{W}_2) = \mathcal{L}(\mathcal{W}_1) \cdot \mathcal{L}(\mathcal{W}_2)$.

Proof sketch. The result follows from the mixed-product property of the Kronecker product:

$$(A \otimes B)(C \otimes D) = (AC) \otimes (BD),$$

and induction on the length of the word. ◀

- **Definition 2.13** (Multiplication by a scalar). Let $\mathcal{W} = (\Sigma, n, \alpha, \mu, \beta)$ be a WFA, and let c be a constant. We define $c \cdot \mathcal{W}$ as the WFA $(\Sigma, n, c \cdot \alpha, \mu, \beta)$.

Multiplying either the initial or final vector by c yields an equivalent construction; we choose to scale the initial vector.

- **Proposition 2.14.** Let $\mathcal{W} = (\Sigma, n, \alpha, \mu, \beta)$ be a WFA, and let c be a constant. Then, we have $\mathcal{L}(c \cdot \mathcal{W}) = c \cdot \mathcal{L}(\mathcal{W})$.

For a correctness proof of these propositions, we refer to the work of Kiefer [15].

These constructions show that the class of weighted languages recognised by WFAs is closed under pointwise addition, pointwise multiplication, and scalar multiplication. In particular, it forms an $\mathbb{R}$ -algebra under these operations.

- **Definition 2.15** (Atomic WFA). Let $w \in \Sigma^*$ and let $c \geq 0$. The atomic automaton $c[w]$ is the weighted language

$$c[w](v) = \begin{cases} c & \text{if } v = w, \\ 0 & \text{otherwise,} \end{cases}$$

for $v \in \Sigma^*$. When $c = 1$ we write simply $[w]$. Note that $c[w]$ is realised by a WFA of size $|w| + 1$ (a chain of states, one per prefix of w , with initial weight c on the first state and final weight 1 on the last), and that $c[w]$ is stochastic whenever $c = 1$.

2.4 Minimisation

In automata theory, it is very natural to ask the question whether we can find minimal automata. Minimal WFAs are desirable both computationally and structurally. They provide the most compact representation of a weighted language and often reveal important algebraic properties of the automaton.

- **Definition 2.16.** A WFA $\mathcal{W}$ is called *minimal* if there does not exist a smaller WFA (with fewer states) that is equivalent to $\mathcal{W}$.

To compute a minimal WFA, we will use concepts called the *forward space* and the *backward space*. These are the vector spaces generated by the transition matrices and the initial and final vectors respectively. The forward and backward spaces capture all behaviours that can be observed from the initial and final vectors under repeated application of the transition matrices. Intuitively, the forward space describes all reachable weighted configurations, while the backward space describes all observable futures. By computing a basis of these vector spaces, we can find a minimal representation of a WFA.

- **Definition 2.17.** Let $\mathcal{W} = (\Sigma, n, \alpha, \mu, \beta)$ be a WFA. The *forward space* of $\mathcal{W}$ is the vector space

$$\mathcal{F} = \text{span}\{\alpha\mu(w) \mid w \in \Sigma^*\} \subseteq \mathbb{R}^{1 \times n}.$$

The *backward space* of $\mathcal{W}$ is the vector space

$$\mathcal{B} = \text{span}\{\mu(w)\beta \mid w \in \Sigma^*\} \subseteq \mathbb{R}^{n \times 1}.$$

- **Definition 2.18** (Forward conjugate). Let $\mathcal{W} = (\Sigma, n, \alpha, \mu, \beta)$ be a WFA. Let F be a matrix whose rows form a basis of the forward space. The *forward conjugate* of $\mathcal{W}$ is the WFA

$$\mathcal{W}_F = (\Sigma, m, \alpha_F, \mu_F, \beta_F)$$

defined by

$$\begin{aligned} \alpha_F &= e_1 \\ F\mu(\sigma) &= \mu_F(\sigma)F \\ \beta_F &= F\beta. \end{aligned}$$

- **Definition 2.19** (Backward conjugate). Let $\mathcal{W} = (\Sigma, n, \alpha, \mu, \beta)$ be a WFA. Let B be a matrix whose columns form a basis of the backward space. The *backward conjugate* of $\mathcal{W}$ is the WFA

$$\mathcal{W}_B = (\Sigma, m, \alpha_B, \mu_B, \beta_B)$$

defined by

$$\begin{aligned} \alpha_B &= \alpha B \\ \mu(\sigma)B &= B\mu_B(\sigma) \\ \beta_B &= e_1. \end{aligned}$$

The next theorem gives us an efficient algorithm to compute a minimal WFA. This was already known in the 1960s [22], but a recent proof is given by Kiefer [15]. Intuitively, the forward conjugate removes redundant unreachable behaviour, while the backward conjugate removes behaviour that does not contribute to the recognised weighted language. Combining both therefore yields a minimal representation.

- **Theorem 2.20.** The forward conjugate of the backward conjugate of a WFA $\mathcal{W}$ is a minimal WFA that is equivalent to $\mathcal{W}$. The same holds for the backward conjugate of the forward conjugate of $\mathcal{W}$. This can be computed in polynomial time.

While we mostly use the linear algebraic perspective on WFAs, sometimes we will fall back on the state-based perspective, in which case the upcoming lemma will be useful.

- **Lemma 2.21.** Let $\mathcal{W}$ be a minimal WFA. Then, for every state q , there exists a word w and a path labelled by w passing through q whose weight is nonzero.

Proof. Write $\mathcal{W} = (\Sigma, n, \alpha, \mu, \beta)$, let $\mathcal{F} = \text{span}\{\alpha\mu(w) \mid w \in \Sigma^*\}$ be its forward space, and let $\mathcal{B} = \text{span}\{\mu(w)\beta \mid w \in \Sigma^*\}$ be its backward space (Definition 2.17).

We first argue that $\dim \mathcal{F} = n$. Indeed, if $\dim \mathcal{F} < n$, then the forward conjugate $\mathcal{W}_F$ (Definition 2.18) would be a WFA with fewer than n states computing the same weighted language as $\mathcal{W}$, contradicting minimality of $\mathcal{W}$. Hence $\mathcal{F} = \mathbb{R}^{1 \times n}$. By a symmetric argument using the backward conjugate, $\dim \mathcal{B} = n$ as well, so $\mathcal{B} = \mathbb{R}^{n \times 1}$.

Fix a state $q \in \{1, \dots, n\}$, and let e_q denote the q -th standard basis vector. Since $\mathcal{F} = \mathbb{R}^{1 \times n}$, we have $e_q \in \mathcal{F}$, so there exist words $u_1, \dots, u_k \in \Sigma^*$ and scalars $c_1, \dots, c_k \in \mathbb{R}$ with

$$\sum_{i=1}^k c_i \alpha\mu(u_i) = e_q.$$

Comparing the q -th coordinate on both sides gives $\sum_{i=1}^k c_i (\alpha\mu(u_i))_q = 1 \neq 0$, so $(\alpha\mu(u))_q \neq 0$ for some $u = u_i$. But $(\alpha\mu(u))_q$ is exactly the sum of the weights of all paths labelled u that lead from an initial state to q ; since this sum is nonzero, at least one such path, say P_1 , has nonzero weight.

By the symmetric argument applied to $\mathcal{B} = \mathbb{R}^{n \times 1}$, there exists a word $v \in \Sigma^*$ with $(\mu(v)\beta)_q \neq 0$, and hence at least one path P_2 labelled v from q to a final state with nonzero weight.

Concatenating P_1 and P_2 yields a path labelled $w = uv$ that passes through q , whose weight is the product of the weights of P_1 and P_2 , and is therefore nonzero. This proves the claim. $\blacktriangleleft$

With the theory of weighted languages and weighted finite automata now in place, we have the necessary vocabulary to formalise the kind of specifications sketched in the introduction. In the next chapter, we put this machinery to use and introduce weighted control improvisation, the central framework of this thesis, and show how the preliminaries developed here allow it to generalise the classical notion of control improvisation.

3 Control Improvisation

In the introduction, we discussed some applications for a framework that allows us to obtain a random sampler that adheres to certain hard and soft requirements. In this chapter, we will formally introduce such a framework, which we will call *weighted control improvisation*, and look at concrete instances of this problem. We will also compare this framework to classical control improvisation, and show that our framework is a generalisation. Algorithms to solve a weighted control improvisation problem will not yet be discussed. Instead, this will be reserved for a later chapter after studying more theory.

3.1 Weighted Control Improvisation

We want to define a framework that allows us to specify hard constraints, that is, requirements that a solution must always satisfy, and soft constraints, that is, requirements that a solution is expected to satisfy.

When talking about probabilistic generators, the hard constraints give a lower and upper bound to the probability that each word is generated. For example, the hard constraints might specify that an improvisation for a drone route must never go out of bounds, and thus assign probability 0 to any such improvisation. They could also assign a higher minimum probability to improvisations that keep a drone close to areas of interest compared to less interesting areas. A natural structure to use is that of weighted finite automata, since they are well-studied and can model complex behaviour even with small automata.

The soft constraints represent the score of each improvisation. Each word is assigned a specific score, and a solution to a weighted control improvisation problem should have an expected score exceeding a specified value. For example, the soft constraints could specify that a drone should complete at least three laps around a specific area on average over all improvisations. Again, it is very natural to use weighted finite automata here.

- **Definition 3.1** (Weighted Control Improvisation). A *weighted control improvisation (WCI) problem* is a tuple (W_L, W_U, S) of weighted finite automata.

A WCI solution is a stochastic WFA $\mathcal{W}$ such that

- $W_L \leq \mathcal{W} \leq W_U$, and
- $\text{weight}(\mathcal{W} \odot S) \geq 1$.

In this thesis, we assume that the languages of all WFAs are non-negative.

We are interested in two main decision problems, namely the synthesis and verification of a WCI problem.

Synthesis: Given a WCI problem, does there exist a solution? If so, construct one.

Verification: Given a WCI problem (W_L, W_U, S) and a stochastic WFA $\mathcal{W}$, is $\mathcal{W}$ a solution to (W_L, W_U, S) ?

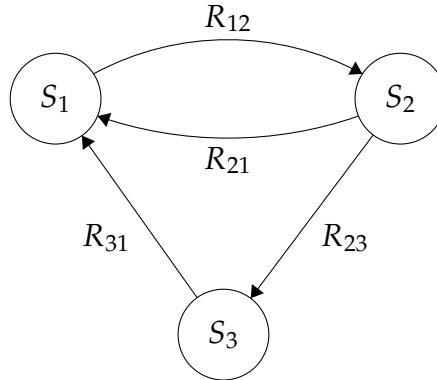
It turns out that both of these problems are, not completely unsurprisingly, undecidable in their most general forms. However, we will see that some practically important cases can still be solved.

Next, we will discuss two examples of WCI problems. In the first example, we study a rail network with several train routes, and a transportation company called the Weighted Railways (WR) that wants to check the tickets of travelling customers in an unpredictable manner.

The WR employs a conductor called Bob, who can travel along their railway network and check the tickets of passengers on the trains. To minimise losses due to fare evasion, they want to ensure that Bob checks a certain minimum number of tickets each day. This could easily be done by always sending Bob along the same busy route. However, passengers would quickly learn to avoid Bob and thus avoid paying for a ticket.

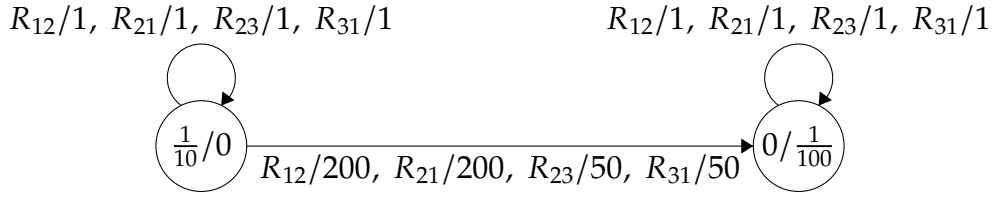
As such, the WR plans to randomly choose a route for Bob each day. Using WCI, they are able to ensure that enough tickets are checked, while maintaining valid routes and a degree of randomness.

For the trial phase, the WR plans to implement WCI on a small isolated part of their rail network, shown below.



This network consists of three stations. Periodically, trains travel each route, all departing at the same time, with each train arriving before the next wave of departures. Each period, the routes R_{12} and R_{21} each serve 200 passengers, while R_{23} and R_{31} each serve only 50 passengers.

Formulating the soft constraints. The WR has determined that checking the tickets of 1 000 passengers on average per shift should be sufficient to deter fare evasion. Assuming Bob starts each day at S_1 , how can we model the number of checked tickets for a route as a WFA? It turns out that this can be done using a WFA with just two states: one representing all station states, and one sink state.



The states here are a single station state and a sink state, with initial and final weights shown as initial/final inside each node. The construction works by exploiting the fact that WFAs sum the weights of all accepting paths. Since none of the four routes distinguish between the stations S_1, S_2, S_3 for the purpose of counting checked tickets, these three states can be merged into a single station state without changing the recognised weighted language. For each route transition, a path has two choices: remain in the station state (weight 1), or defect to the sink (weight equal to the passenger count). Only paths that reach the sink contribute to the final weight, since the station state has final weight 0. A path that defects after reading the i -th symbol σ_i has weight

$$\frac{1}{10} \cdot 1^{i-1} \cdot p(\sigma_i) \cdot 1^{n-i} \cdot \frac{1}{100} = \frac{p(\sigma_i)}{1\,000},$$

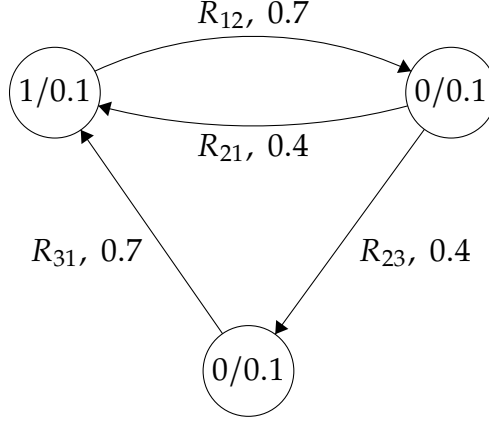
where $p(\sigma_i)$ is the passenger count for route σ_i and n is the total length of the shift. Summing over all defection points gives

$$\mathcal{L}(S)(w) = \frac{1}{1\,000} \sum_{i=1}^n p(\sigma_i) = \frac{\text{passengers}(w)}{1\,000}.$$

The soft constraint $\text{weight}(\mathcal{W} \odot S) \geq 1$ therefore expresses that the expected number of passengers checked per shift is at least 1 000.

Formulating the hard constraints. We also need to ensure that generated routes are operationally valid. Furthermore, to prevent passengers

from learning Bob’s exact schedule, every feasible route from every station should receive some non-zero probability. We now construct a WFA representing the lower bounds.



In this lower-bound automaton, Bob is guaranteed to start at S_1 . From there, he has a probability of at least 0.1 to immediately end his shift, and a probability of at least 0.7 to take the R_{12} train to S_2 . From S_2 , he again has a probability of at least 0.1 to end his shift, at least 0.4 to take R_{21} back to S_1 , and at least 0.4 to take R_{23} to S_3 . From S_3 , he has at least 0.1 to end his shift and at least 0.7 to take R_{31} back to S_1 .

Note that the weights at each station do not sum to 1: for example, $0.1 + 0.7 = 0.8$ at S_1 . This is intentional. The automaton W_L specifies *minimum* probabilities for each choice; the remaining probability mass is left for the improviser to distribute freely, subject to the upper bound W_U . We can obtain W_U from W_L by setting each non-zero weight to 1, which leaves the topology of the network intact while imposing no additional restriction beyond route validity.

Now we can use these WFAs as the input to a WCI problem. A solution would give us a stochastic WFA that generates valid routes, respects the given minimum probabilities, and checks at least 1 000 passengers on average. We do not yet have the tools to solve such a WCI problem; these will be developed in the next chapter. Before that, we compare our WCI framework to that of classical control improvisation.

3.2 Classical Control Improvisation

Weighted control improvisation is designed to generalise the classical notion of control improvisation. The two frameworks share the same basic intuition: one wants to generate words randomly while still enforcing hard constraints and respecting a notion of quality or desirability. The difference is that classical control improvisation is fundamentally Boolean and length-bounded, whereas weighted control improvisation is quantitative and can naturally work over arbitrarily long improvisations. This additional generality is exactly what makes the weighted framework suitable for the examples considered in this thesis.

We first recall the classical setting. A *classical control improvisation instance* is a tuple

$$\mathcal{I} = (H, S, \epsilon, \lambda, \rho, n),$$

where $H, S \subseteq \Sigma^*$ are regular languages, $0 \leq \epsilon \leq 1$, $0 \leq \lambda \leq \rho \leq 1$, and $n \in \mathbb{N}$. A probabilistic generator G is a *solution* to $\mathcal{I}$ if the following conditions hold:

- every generated word satisfies the hard constraint and has length at most n :

$$\text{supp}(G) \subseteq H \cap \Sigma^{\leq n};$$

- every admissible word receives probability between the prescribed lower and upper bounds:

$$\forall w \in H \cap \Sigma^{\leq n}, \quad \lambda \leq G(w) \leq \rho;$$

- the soft constraint is satisfied with sufficiently high probability:

$$\Pr_G[S] \geq 1 - \epsilon.$$

The hard constraint specifies which words may be produced at all. The lower bound prevents the generator from ignoring any individual admissible word, while the upper bound ensures the distribution remains sufficiently randomised. The length bound n is needed to keep the candidate set finite.

Weighted control improvisation extends this picture by replacing languages with weighted automata. Instead of merely recording whether a word is allowed or disallowed, a weighted automaton can assign an arbitrary non-negative real value to each word. This makes it possible to express quantitative constraints directly: the hard and soft requirements are no longer limited to Boolean acceptance conditions, but can

describe costs, rewards, frequencies, resource consumption, or expected performance measures.

The reduction from classical CI to WCI is straightforward. Let $\mathcal{I} = (H, S, \epsilon, \lambda, \rho, n)$ be a classical instance. We construct a weighted instance (W_L, W_U, T) as follows. Define

$$W_L(w) = \begin{cases} \lambda & \text{if } w \in H \cap \Sigma^{\leq n}, \\ 0 & \text{otherwise,} \end{cases} \quad W_U(w) = \begin{cases} \rho & \text{if } w \in H \cap \Sigma^{\leq n}, \\ 0 & \text{otherwise,} \end{cases}$$

$$T(w) = \begin{cases} \frac{1}{1 - \epsilon} & \text{if } w \in S, \\ 0 & \text{otherwise.} \end{cases}$$

If G is a stochastic WFA, the condition $W_L \leq G \leq W_U$ enforces the pointwise probability bounds on every admissible word and forbids probability mass on words outside $H \cap \Sigma^{\leq n}$. For the soft constraint, we have

$$\text{weight}(G \odot T) = \sum_{w \in \Sigma^*} G(w) T(w) = \frac{1}{1 - \epsilon} \sum_{w \in S} G(w) = \frac{\Pr_G[S]}{1 - \epsilon},$$

so $\text{weight}(G \odot T) \geq 1$ if and only if $\Pr_G[S] \geq 1 - \epsilon$. Therefore G is a classical CI solution to $\mathcal{I}$ if and only if it is a WCI solution to (W_L, W_U, T) .

This shows that classical control improvisation is contained in the weighted framework as a special case. The reduction is also conceptually revealing. The classical parameters λ and ρ are uniform: the same bounds apply to every admissible word. In WCI, by contrast, the bounds can vary from word to word. One may assign a different lower and upper bound to each individual word simply by choosing suitable automata W_L and W_U . This is a substantial increase in expressive power, allowing the model to distinguish between words that are admissible but not equally desirable, and to encode fine-grained quantitative preferences that a single global probability interval cannot capture.

This pointwise flexibility is particularly important in applications. In the rail-network example, one route may be much more valuable than another because it passes through a busier station, while a different route may still be valid but less useful. A classical improviser can only say that a route is allowed or not, and then impose the same probability bounds on all allowed routes. A weighted improviser can instead attach different lower and upper bounds to different routes, reflecting their relative importance directly in the model.

Another important difference is the treatment of word length. The length bound n is part of the classical definition and plays a central role in its theory: without it, the candidate set would be infinite and the

probability normalisation condition would require careful treatment. Weighted control improvisation does not need such a bound. A WFA can assign weights to words of arbitrary length, and whether the resulting language has finite total weight is characterised by the spectral radius of the aggregate transition matrix (Theorem 4.8), a condition checkable in polynomial time. This makes it possible to model processes with no natural finite horizon (such as Bob’s shifts, which may in principle be of any length), without imposing an artificial cap.

The ability to handle unbounded improvisations changes the kinds of systems that can be modelled. Classical CI is best suited to bounded generation tasks where all candidate words have a prescribed maximum length. Weighted CI can represent processes that may continue for an arbitrary number of steps while still enforcing quantitative constraints on the generated behaviour.

Despite these advantages, weighted CI also has drawbacks worth acknowledging. The increased flexibility comes at a specification cost: designing the automata W_L , W_U , and S requires more domain knowledge than simply choosing scalar parameters λ , ρ , and ϵ . In practice, useful lower- and upper-bound WFAs often have a structure that already resembles the solution itself. A second disadvantage is computational: classical CI is solvable in polynomial time for bounded specifications, while WCI synthesis and verification are undecidable in general. The additional quantitative structure means one must reason about expectations, products of weighted automata, and potentially infinite behaviour—Operations that are harder than their Boolean counterparts and undecidable in the general case.

To summarise, the classical theory is recovered from the weighted one by restricting weights to Boolean values and encoding the classical uniform bounds and length bound in weighted automata. The weighted framework goes beyond that in three essential ways. First, it allows word-dependent lower and upper probability bounds instead of a single global interval. Second, it allows improvisations of arbitrary length rather than requiring a fixed maximum. Third, it permits genuine quantitative objectives such as expected scores or costs, which classical CI cannot express. For these reasons, weighted control improvisation should be viewed as a strict and useful generalisation of the classical notion.

3.3 A Randomised Park-Route Example

To illustrate the usefulness of weighted control improvisation in a setting where randomness has a clear practical role, consider a large public park that offers visitors a set of suggested walking routes. The park contains several points of interest, including an entrance plaza, a coffee shop, a playground, a lake, a botanical garden, a viewpoint, and an exit. Visitors enter at the main entrance and follow a route through some of these destinations before leaving. The park management would like to generate suggested routes automatically, but not deterministically. If every visitor were directed along the same path, the coffee shop, the viewpoint, and the lake promenade would quickly become crowded while less popular areas remained unused. The route generator should therefore be randomised. At the same time, routes must be valid, must stay within the park, and should satisfy certain quantitative properties.

This is a natural application for weighted control improvisation. If the route generator were predictable, visitors would cluster on the same scenic trail, and the resulting congestion would reduce both visitor comfort and the effectiveness of the park's facilities. By producing routes randomly while still controlling the probability of each route, the park can encourage variety and prevent overcrowding at any single destination. In this sense, randomness is not merely an abstract requirement but an active tool for improving the quality of the system.

Suppose the park is modelled as a directed graph whose vertices are the main destinations and whose edges are the walkable paths connecting them. The visitor starts at the entrance plaza E and must eventually leave through the exit X . Between these two points, the route may visit the coffee shop C , the playground P , the lake L , the botanical garden B , and the viewpoint V .

The alphabet is $\Sigma = \{EC, EP, CL, CB, PL, PB, LV, LX, BV, BX, VX\}$, where each symbol denotes the edge traversed. A word over Σ then represents a sequence of steps through the park.

Formulating the constraints. The upper-bound automaton W_U encodes the park layout. It has one state per destination plus a sink for invalid transitions, with initial weight 1 on E , final weight 1 on X , and transition weight 1 on each valid edge. Routes that attempt to leave through a service entrance, backtrack along a one-way path, or skip a required connection receive weight 0.

The lower- and upper-bound automata are particularly useful here. Different routes should not all have the same probability, since some routes are more important than others from the park's perspective. Routes that visit the coffee shop may be assigned a higher lower bound if the park wants to ensure the shop receives enough foot traffic to

remain viable. Routes through the botanical garden may also be given a higher minimum probability, since the park may wish to encourage visitors to discover less obvious attractions. At the same time, no single route should dominate the distribution, so upper bounds can be used to prevent any one path from becoming too likely.

For instance, the route $E \rightarrow C \rightarrow L \rightarrow X$ might be given a moderate lower bound because it is short and pleasant, but a relatively low upper bound because it would otherwise become too common. The route $E \rightarrow C \rightarrow B \rightarrow V \rightarrow X$ might be assigned a higher lower bound, since the park would like to guide visitors towards the quieter garden and the scenic viewpoint. Meanwhile, a more direct route may still be allowed, but only with a small lower bound, since it contributes little to the visitor experience. A classical CI improviser could only say that a route is allowed or not, and then impose the same probability bounds on every allowed route. A WCI improviser can instead encode these distinctions directly.

The scoring automaton S measures the attractiveness of a suggested route. One natural interpretation is that each destination visited contributes a reward: the coffee shop contributes 1 point, the playground 1 point, the lake promenade 2 points, the botanical garden 3 points, and the viewpoint 4 points. The score of a route is then the sum of rewards collected along it. Using the same two-copy construction as in the railway example, a station copy that accumulates no weight and a sink copy that is entered when a destination is reached, we can build a WFA S such that $\mathcal{L}(S)(w)$ equals the total score of route w divided by a normalisation constant θ . Setting $\theta = 5$ makes the soft constraint $\text{weight}(\mathcal{W} \odot S) \geq 1$ say that the expected route score is at least 5 points. A route through only the lake earns 2 points, which is unlikely to drive the expectation above the threshold on its own; a route through the garden and viewpoint earns 7 points and contributes substantially. The improviser is therefore nudged towards more scenic routes without being forced to generate them deterministically.

A solution to this WCI problem is a probabilistic route generator that produces only valid routes, distributes probability according to the specified bounds, and achieves the required expected attractiveness. Because the routes are randomised, repeated visitors will not see the same suggestion every day. Because the randomness is controlled, the park can still ensure that important destinations receive enough attention.

Potential extensions. We really can get creative with our input automata here. For example, we can model that walking towards the coffee shop becomes more likely after being in the park for a certain amount of time, or that you are less likely to visit the coffee shop after having

already been there.

The park setting also illustrates why allowing arbitrarily long routes is useful. Some visitors may want a brief walk that ends after a few destinations, while others may prefer a longer tour that loops through multiple areas. There is no natural upper bound on route length: the number of stops a visitor makes can depend on their mood, the weather, or the time available. A fixed length bound would therefore be artificial. Weighted control improvisation does not require one; it can model valid routes of any length while still enforcing the probability bounds and the expected quality requirement.

In summary, this example demonstrates the practical value of weighted control improvisation in a setting where randomness is itself a design objective. Randomised routes spread visitors across multiple destinations, reduce crowding, and provide variety across repeated visits. The lower and upper bounds allow the park to control how often each route appears, the soft constraint measures how attractive a route is on average, and the absence of a fixed length bound makes it possible to model routes of varying duration without artificial restrictions.

4 Weighted Finite Automata

In this chapter, we will study and develop several concepts which will be important for solving a WCI problem. We start by computing the weight of the language of a WFA. We follow this by studying the containment problem for WFAs (which instantiates to pointwise domination of the languages).

4.1 Computing the Total Weight

In the minimisation algorithm, we used a *change of basis*. A change of basis replaces a matrix M by a similar matrix

$$\hat{M} = P^{-1}MP,$$

where P is invertible. The powers of similar matrices satisfy

$$\hat{M}^n = PM^nP^{-1},$$

so understanding the powers of $\hat{M}$ yields information about the asymptotic behaviour of M . A useful change of basis that we can apply here is the Jordan normal form.

► **Definition 4.1** (Jordan Normal Form). Let A be a square matrix. A matrix J is called a *Jordan normal form* of A if J is block diagonal and similar to A , i.e.

$$A = PJP^{-1}$$

for some invertible matrix P , and J has the form

$$J = \text{diag}(J_1, J_2, \dots, J_k),$$

where each block J_i is a *Jordan block* corresponding to an eigenvalue λ_i and has the form

$$J_i = \begin{pmatrix} \lambda_i & 1 & 0 & \cdots & 0 \\ 0 & \lambda_i & 1 & \cdots & 0 \\ 0 & 0 & \lambda_i & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \lambda_i \end{pmatrix}.$$

With the following lemma, we will be able to easily characterise the behaviour of long words, by reducing the powers of a matrix to powers of individual matrix entries.

► **Lemma 4.2.** Let $J_k(\lambda)$ be the $k \times k$ Jordan block with eigenvalue λ , i.e.

$$J_k(\lambda) = \begin{pmatrix} \lambda & 1 & 0 & \cdots & 0 \\ 0 & \lambda & 1 & \cdots & 0 \\ 0 & 0 & \lambda & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \lambda \end{pmatrix}.$$

Then for every $n \geq 0$,

$$J_k(\lambda)^n = \begin{pmatrix} \lambda^n & \binom{n}{1}\lambda^{n-1} & \binom{n}{2}\lambda^{n-2} & \cdots & \binom{n}{k-1}\lambda^{n-k+1} \\ 0 & \lambda^n & \binom{n}{1}\lambda^{n-1} & \cdots & \binom{n}{k-2}\lambda^{n-k+2} \\ 0 & 0 & \lambda^n & \cdots & \binom{n}{k-3}\lambda^{n-k+3} \\ \vdots & \vdots & \vdots & \ddots & \binom{n}{1}\lambda^{n-1} \\ 0 & 0 & 0 & \cdots & \lambda^n \end{pmatrix}.$$

Equivalently,

$$(J_k(\lambda)^n)_{ij} = \begin{cases} \binom{n}{j-i}\lambda^{n-(j-i)} & i \leq j, \\ 0 & i > j. \end{cases}$$

Proof. The formula for the powers of a Jordan block is standard and follows from the binomial expansion of $(\lambda I + N)^n$, where N is the nilpotent part of the block (see, e.g., [13]). ◀

In practice one can use one of the many linear algebra libraries to directly compute the eigenvalues.

Using this lemma, we can see that the powers of a matrix consisting of Jordan blocks will approach zero if and only if the eigenvalue of greatest magnitude (also known as the spectral radius) has a magnitude smaller than one.

► **Definition 4.3** (Spectral radius). Let $\lambda_1, \dots, \lambda_n$ be the eigenvalues of a matrix $A \in \mathbb{R}^{n \times n}$. The *spectral radius* of A is defined as

$$\rho(A) = \max \{|\lambda_1|, \dots, |\lambda_n|\}.$$

► **Lemma 4.4** (Neumann series). Let A be a square real-valued matrix. Then,

$$\sum_{k=0}^{\infty} A^k = (I - A)^{-1}$$

if and only if $\rho(A) < 1$.

In the following example, we will use the direct computation of the powers of the transition matrix to derive a classical result about the Fibonacci sequence.

► **Example 4.5.** Let $\mathcal{W}$ be the WFA of example 2.6. The eigenvalues of the transition matrix $\mu(a)$ are

$$\lambda_1 = \frac{1 + \sqrt{5}}{2}, \quad \lambda_2 = \frac{1 - \sqrt{5}}{2}.$$

Since the eigenvalues are distinct, $\mu(a)$ is diagonalizable, and its Jordan normal form is

$$J = \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix}.$$

Let

$$P = \begin{pmatrix} \lambda_1 & \lambda_2 \\ 1 & 1 \end{pmatrix}.$$

Then

$$\mu(a) = PJP^{-1}.$$

For the weight of a^n we obtain

$$\alpha \mu(a)^n \beta = \alpha P J^n P^{-1} \beta.$$

Define new vectors

$$\alpha' = \alpha P, \quad \beta' = P^{-1} \beta.$$

This gives

$$\alpha' = (\lambda_1 \quad \lambda_2), \quad \beta' = \frac{1}{\lambda_1 - \lambda_2} \begin{pmatrix} -\lambda_2 \\ \lambda_1 \end{pmatrix}.$$

Since

$$J^n = \begin{pmatrix} \lambda_1^n & 0 \\ 0 & \lambda_2^n \end{pmatrix},$$

we obtain

$$\begin{aligned} \mathcal{L}(\mathcal{W})(a^n) &= (\lambda_1 \quad \lambda_2) \begin{pmatrix} \lambda_1^n & 0 \\ 0 & \lambda_2^n \end{pmatrix} \frac{1}{\lambda_1 - \lambda_2} \begin{pmatrix} 1 \\ -1 \end{pmatrix} \\ &= \frac{1}{\lambda_1 - \lambda_2} (\lambda_1^{n+1} - \lambda_2^{n+1}). \end{aligned}$$

Since $\lambda_1 - \lambda_2 = \sqrt{5}$, this yields the closed form

$$\mathcal{L}(\mathcal{W})(a^n) = \frac{\lambda_1^n - \lambda_2^n}{\sqrt{5}}.$$

Thus the WFA computes the Fibonacci numbers and the Jordan decomposition directly yields Binet's formula. Since $|\lambda_2| < 1$, the asymptotic growth is governed by λ_1 , showing that the Fibonacci numbers grow exponentially at rate λ_1^n .

Although the Jordan normal form gives a conceptual characterisation of the condition $\rho(M) < 1$, computing Jordan forms numerically is often unstable. The following equivalent criterion can be better suited for algorithmic verification.

► **Proposition 4.6** (Lyapunov criterion for $\rho(A) < 1$). Let $A \in \mathbb{R}^{n \times n}$. The following are equivalent:

1. The spectral radius satisfies $\rho(A) < 1$.
2. There exists a symmetric positive definite matrix $P > 0$ such that

$$A^\top P A - P = -I.$$

3. There exists a symmetric positive definite matrix $P > 0$ such that

$$A^\top P A - P < 0.$$

Proof. A proof of the equivalence of conditions for $\rho(A) < 1$ using Lyapunov's method can be found in Horn and Johnson [12]. ◀

The Jordan Normal Form (and spectral radius) of a transition matrix can also be used to compute the total weight of a WFA. Notice the use of the singular form of matrix; we will first have to reduce the problem to the case with only one input symbol.

► **Lemma 4.7.** Let $\mathcal{W} = (\Sigma, n, \alpha, \mu, \beta)$ be a non-negative WFA. Let $M = \sum_{\sigma \in \Sigma} \mu(\sigma)$. Then,

$$\text{weight}(\mathcal{W}) = \sum_{k=0}^{\infty} \alpha M^k \beta.$$

Proof. We will use induction to show that $\sum_{w \in \Sigma^k} \mu(w) = M^k$ for all k . We have $\mu(\varepsilon) = M^0$ for the base case. For the inductive step, we have

$$\begin{aligned} \sum_{w \in \Sigma^{k+1}} \mu(w) &= \sum_{\sigma \in \Sigma} \mu(\sigma) \left(\sum_{w \in \Sigma^k} \mu(w) \right) \\ &= \sum_{\sigma \in \Sigma} \mu(\sigma) M^k \\ &= M \cdot M^k \\ &= M^{k+1}. \end{aligned}$$

Then we have

$$\begin{aligned}
\text{weight}(\mathcal{W}) &= \sum_{w \in \Sigma^*} \alpha \mu(w) \beta \\
&= \sum_{k=0}^{\infty} \sum_{w \in \Sigma^k} \alpha \mu(w) \beta \\
&= \alpha \left(\sum_{k=0}^{\infty} \sum_{w \in \Sigma^k} \mu(w) \right) \beta \\
&= \alpha \left(\sum_{k=0}^{\infty} M^k \right) \beta \\
&= \sum_{k=0}^{\infty} \alpha M^k \beta,
\end{aligned}$$

showing the desired equality. ◀

► **Theorem 4.8.** Let $\mathcal{W} = (\Sigma, n, \alpha, \mu, \beta)$ be a WFA. Then, we can decide whether $\text{weight}(\mathcal{W})$ converges in polynomial time, and if it does, we can compute it in polynomial time.

This problem has been studied in [5], but we explore an alternative (but similar) proof.

Proof. Define

$$M = \sum_{\sigma \in \Sigma} \mu(\sigma).$$

By Lemma 4.7,

$$\text{weight}(\mathcal{W}) = \sum_{k=0}^{\infty} \alpha M^k \beta.$$

Hence the problem reduces to studying the convergence of the matrix series

$$\sum_{k=0}^{\infty} \alpha M^k \beta.$$

We now minimise the corresponding one-symbol WFA. Let

$$(\alpha', M', \beta')$$

denote the resulting minimal representation. By construction,

$$\alpha M^k \beta = \alpha' (M')^k \beta'$$

for every $k \geq 0$.

We claim that the series converges if and only if

$$\rho(M') < 1,$$

where $\rho(M')$ denotes the spectral radius of M' .

First assume that

$$\rho(M') < 1.$$

Then the Neumann series converges:

$$\sum_{k=0}^{\infty} (M')^k = (I - M')^{-1}.$$

Consequently,

$$\sum_{k=0}^{\infty} \alpha' (M')^k \beta'$$

converges as well, and therefore $\text{weight}(\mathcal{W})$ exists.

Conversely, assume that

$$\rho(M') \geq 1.$$

Write M' in Jordan normal form:

$$M' = PJP^{-1},$$

where J is block diagonal with Jordan blocks

$$J_1, \dots, J_r.$$

Define

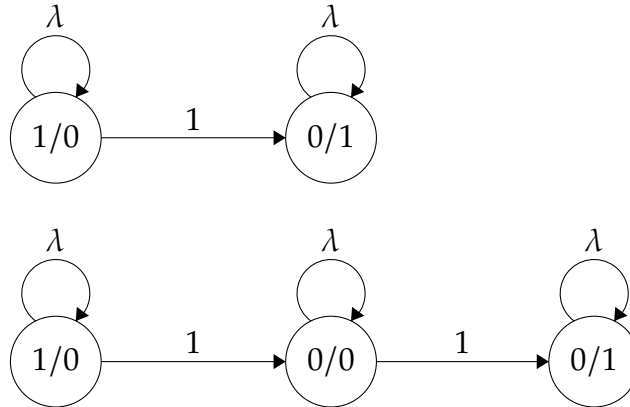
$$\tilde{\alpha} = \alpha' P, \quad \tilde{\beta} = P^{-1} \beta'.$$

Let J_b be a Jordan block corresponding to an eigenvalue λ satisfying

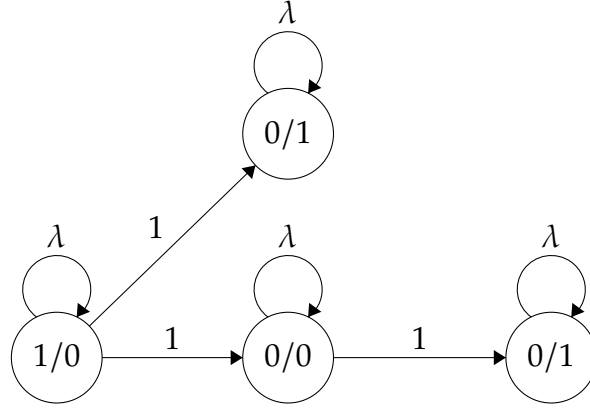
$$|\lambda| = \rho(M').$$

Suppose that J_b has size m .

First, we will show that each eigenvalue of M' is unique. Consider an example where this is not the case:



Note that this is one WFA, which is disconnected. This WFA is not minimal, since the following WFA induces the same language:



We can see this by noting that all we are doing is delaying the choice of entering the top branch or the bottom branch. This contradicts the WFA being minimal by construction.

Since the automaton is minimal, every state is reachable and observable. It follows that the first coordinate of the component of $\tilde{\alpha}$ corresponding to J_b is nonzero, and the last coordinate of the corresponding component of $\tilde{\beta}$ is nonzero.

By the formula for powers of Jordan blocks, the upper-right entry of

$$J_b^k$$

is

$$\binom{k}{m-1} \lambda^{k-m+1}.$$

Hence this entry grows asymptotically like

$$k^{m-1} \lambda^k.$$

Therefore,

$$\alpha'(M')^k \beta' = \tilde{\alpha} J^k \tilde{\beta}$$

contains a nonzero term with asymptotic growth

$$k^{m-1} \lambda^k.$$

If $|\lambda| > 1$, then these terms grow exponentially and therefore do not converge to zero.

If $|\lambda| = 1$, then the terms grow polynomially (or remain bounded away from zero when $m = 1$), and again do not converge to zero.

In both cases, the series

$$\sum_{k=0}^{\infty} \alpha'(M')^k \beta'$$

diverges.

We conclude that

$$\text{weight}(\mathcal{W}) \text{ exists} \iff \rho(M') < 1.$$

Finally, the spectral radius can be checked in polynomial time using, for example, the Lyapunov criterion. If $\rho(M') < 1$, then

$$\text{weight}(\mathcal{W}) = \sum_{k=0}^{\infty} \alpha'(M')^k \beta' = \alpha'(I - M')^{-1} \beta',$$

which can also be computed in polynomial time. $\blacktriangleleft$

In summary, we first minimise the WFA, then check if the spectral radius of the sum of transition matrices is less than one (which we can do efficiently using the Lyapunov equation).

The previous theorem gives a complete characterisation of when a WFA defines a finite weighted language. In the special case where the total weight equals 1, the automaton admits a probabilistic interpretation.

► **Theorem 4.9.** Let $\mathcal{W} = (\Sigma, n, \alpha, \mu, \beta)$ be a WFA. If $\text{weight}(\mathcal{W}) = 1$, then there exists an equally sized stochastic WFA $\mathcal{P}$ such that $\mathcal{L}(\mathcal{W}) = \mathcal{L}(\mathcal{P})$.

Proof. Define

$$M = \sum_{\sigma \in \Sigma} \mu(\sigma).$$

Since $\text{weight}(\mathcal{W}) = 1$, the previous theorem implies that

$$\rho(M) < 1.$$

Hence the series

$$v = \sum_{k=0}^{\infty} M^k \beta$$

converges.

Observe that

$$v = Mv + \beta.$$

Since $\mathcal{W}$ is non-negative, every entry of v is strictly positive after removing unreachable states. Define the diagonal matrix

$$Q = \text{diag}(v_1, \dots, v_n).$$

We now define a new WFA

$$\mathcal{P} = (\Sigma, n, \alpha_Q, \mu_Q, \beta_Q)$$

by

$$\begin{aligned}\alpha_Q &= \alpha Q, \\ \mu_Q(\sigma) &= Q^{-1} \mu(\sigma) Q, \\ \beta_Q &= Q^{-1} \beta.\end{aligned}$$

Since this construction is a change of basis,

$$\mathcal{L}(\mathcal{P}) = \mathcal{L}(\mathcal{W}).$$

It remains to verify that $\mathcal{P}$ is stochastic.

First,

$$\sum_{i=1}^n (\alpha_Q)_i = \alpha v = \sum_{k=0}^{\infty} \alpha M^k \beta = \text{weight}(\mathcal{W}) = 1.$$

Next, for every state i ,

$$\begin{aligned}\sum_{\sigma \in \Sigma} \sum_{j=1}^n (\mu_Q(\sigma))_{ij} + (\beta_Q)_i &= \frac{1}{v_i} \left(\sum_{j=1}^n M_{ij} v_j + \beta_i \right) \\ &= \frac{1}{v_i} (Mv + \beta)_i \\ &= \frac{v_i}{v_i} \\ &= 1.\end{aligned}$$

Thus the outgoing transition weights together with the final weight form a probability distribution at every state. Hence $\mathcal{P}$ is stochastic. ◀

Theorem 4.8 and Theorem 4.9 show that we can efficiently decide whether a WFA has finite total weight and, when that weight equals one, that it is equivalent to some stochastic WFA of the same size. This is already enough to know that such a $\mathcal{W}$ defines a genuine probability distribution over Σ^* . In the next section, we show that this translation is unnecessary: a non-negative WFA with total weight one can be sampled from directly, without ever constructing $\mathcal{P}$, by computing remaining weights on the fly.

4.2 Sampling from Non-Stochastic WFAs

A stochastic WFA can be sampled from directly, since its transition weights and final weights already form, state by state, a valid probability distribution (Section 2.2). A general non-negative WFA $\mathcal{W} = (\Sigma, n, \alpha, \mu, \beta)$ with $\text{weight}(\mathcal{W}) = 1$ need not have this property: the outgoing weights at a state need not sum to 1, or even to a value ≤ 1 . Nevertheless, such a $\mathcal{W}$ still defines a genuine probability distribution over Σ^* , and it is equivalent to some stochastic WFA $\mathcal{P}$ of the same size. Rather than explicitly constructing $\mathcal{P}$, we can sample directly from $\mathcal{W}$ by computing, at each step, the probability of each symbol as the *ratio* of remaining total weight with that symbol consumed to the total weight remaining before consuming it.

Assume $M = \sum_{\sigma \in \Sigma} \mu(\sigma)$ satisfies $\rho(M) < 1$, so that $N := (I - M)^{-1}$ exists. For a row vector $v \in \mathbb{R}^{1 \times n}$, define the *remaining weight* of v as

$$R(v) = vN\beta = \sum_{k=0}^{\infty} vM^k\beta,$$

which is precisely the total weight assigned by $\mathcal{W}$ to all continuations, when the automaton's running (unnormalised) state vector is v . In particular $R(\alpha) = \text{weight}(\mathcal{W}) = 1$.

The key observation is that, starting from running vector v , the process of generating the next symbol (or terminating) partitions the remaining weight $R(v)$ additively:

$$R(v) = v\beta + \sum_{\sigma \in \Sigma} R(v\mu(\sigma)).$$

Indeed, $v\beta$ is exactly the weight contributed by stopping immediately, and $v\mu(\sigma)$ is the running vector after reading σ , so $R(v\mu(\sigma))$ is the weight contributed by all continuations beginning with σ . Since $\mathcal{W}$ is non-negative, every term on the right-hand side is non-negative, and they sum to $R(v)$; this is exactly a probability distribution over the choice “stop”, “read a_1 ”, $\dots$, “read $a_{|\Sigma|}$ ”, once each term is divided by $R(v)$.

This suggests the following sampling procedure. Fix a non-negative WFA $\mathcal{W} = (\Sigma, n, \alpha, \mu, \beta)$ with $\rho(M) < 1$ and $\text{weight}(\mathcal{W}) = 1$, and precompute $N = (I - M)^{-1}$ once, up front. Initialise a running vector $v \leftarrow \alpha$ and an empty word $w \leftarrow \varepsilon$. Then, repeat the following steps:

1. Compute the remaining weight $r \leftarrow vN\beta$.
2. Compute the stopping probability $p_{\text{stop}} \leftarrow \frac{v\beta}{r}$ and, for each $\sigma \in \Sigma$, the continuation probability $p_{\sigma} \leftarrow \frac{(v\mu(\sigma))N\beta}{r}$. By the identity above, $p_{\text{stop}} + \sum_{\sigma \in \Sigma} p_{\sigma} = 1$.

3. Sample an outcome $x \in \{\text{stop}\} \cup \Sigma$ according to the distribution $(p_{\text{stop}}, (p_\sigma)_{\sigma \in \Sigma})$.
4. If $x = \text{stop}$, terminate and output w .
5. Otherwise, append the symbol: $w \leftarrow wx$, update the running vector $v \leftarrow v \mu(x)$, and return to step 1.

► **Proposition 4.10.** The above procedure terminates with probability 1 and outputs each word $w \in \Sigma^*$ with probability exactly $\mathcal{L}(\mathcal{W})(w)$.

Proof. Throughout, write $M = \sum_{\sigma \in \Sigma} \mu(\sigma)$ and $N = (I - M)^{-1}$, and recall $R(v) = vN\beta$ for a row vector v .

Step 1: an exact identity for R . From $N = (I - M)^{-1}$ we get $N - MN = I$, i.e. $N = I + MN$. Multiplying on the right by β and on the left by v gives

$$vN\beta = v\beta + vMN\beta.$$

Since $vM = \sum_{\sigma \in \Sigma} v \mu(\sigma)$, the second term expands as

$$vMN\beta = \sum_{\sigma \in \Sigma} (v \mu(\sigma))N\beta = \sum_{\sigma \in \Sigma} R(v \mu(\sigma)).$$

Combining both displays yields the exact identity

$$R(v) = v\beta + \sum_{\sigma \in \Sigma} R(v \mu(\sigma)) \quad \text{for every } v \in \mathbb{R}^{1 \times n}. \quad (*)$$

Dividing (*) by $R(v)$ shows that, when the running vector is v , the stopping probability $v\beta/R(v)$ together with the continuation probabilities $R(v\mu(\sigma))/R(v)$ for $\sigma \in \Sigma$ sum to 1; this is precisely the distribution used in Step 2 of the sampling procedure.

Step 2: the output distribution. Fix a word $w = w_1 w_2 \cdots w_k \in \Sigma^*$, and define the sequence of running vectors

$$v_0 = \alpha, \quad v_i = v_{i-1} \mu(w_i) \quad (1 \leq i \leq k),$$

so that $v_i = \alpha \mu(w_1 \cdots w_i)$. By construction, the probability that the procedure produces exactly the prefix $w_1, \dots, w_k$ and then stops is the product, over each step, of the probability of choosing w_i at running vector v_{i-1} , times the probability of stopping at running vector v_k :

$$\Pr[\text{output} = w] = \left(\prod_{i=1}^k \frac{R(v_i)}{R(v_{i-1})} \right) \cdot \frac{v_k \beta}{R(v_k)}.$$

The product telescopes to $R(v_k)/R(v_0)$, so

$$\Pr[\text{output} = w] = \frac{R(v_k)}{R(v_0)} \cdot \frac{v_k \beta}{R(v_k)} = \frac{v_k \beta}{R(v_0)} = \frac{v_k \beta}{R(\alpha)}.$$

Since $\text{weight}(\mathcal{W}) = 1$, we have $R(\alpha) = \alpha N \beta = \sum_{j=0}^{\infty} \alpha M^j \beta = \text{weight}(\mathcal{W}) = 1$. Hence

$$\Pr[\text{output} = w] = v_k \beta = \alpha \mu(w) \beta = \mathcal{L}(\mathcal{W})(w),$$

as required.

Step 3: termination with probability 1. The events $\{\text{output} = w\}$ for distinct $w \in \Sigma^*$ are pairwise disjoint, since the procedure outputs at most one word. By Step 2,

$$\begin{aligned} \Pr[\text{procedure terminates}] &= \sum_{w \in \Sigma^*} \Pr[\text{output} = w] \\ &= \sum_{w \in \Sigma^*} \mathcal{L}(\mathcal{W})(w) \\ &= \text{weight}(\mathcal{W}) \\ &= 1, \end{aligned}$$

using non-negativity of $\mathcal{W}$ to justify summing the disjoint-event probabilities term by term. Thus the procedure terminates almost surely, and by Step 2 its output distribution coincides exactly with $\mathcal{L}(\mathcal{W})$. ◀

Having established that sampling from a non-negative WFA with finite total weight can be carried out exactly and efficiently, we have completed our toolkit for treating a WFA as a genuine probabilistic generator: we can decide and compute its total weight (Section 4.1), and, when that weight equals one, sample from it directly without first converting it into stochastic form. What remains is to compare such generators against one another and against fixed bounds, which is exactly the kind of question that arises when checking whether a candidate improviser respects the hard constraints of a WCI instance. We turn to this comparison problem, known as containment, in the next section.

4.3 The Containment Problem

In control improvisation, one typically imposes lower and upper bounds on the probability of generating a particular word. This naturally leads to the comparison of weighted automata: given two WFAs, can we decide whether one always assigns a smaller weight than the other?

- **Definition 4.11** (Containment problem for WFAs). Let $\mathcal{W}_1$ and $\mathcal{W}_2$ be WFAs over the same alphabet Σ . The *containment problem* asks whether

$$\mathcal{L}(\mathcal{W}_1)(w) \leq \mathcal{L}(\mathcal{W}_2)(w) \quad \text{for all } w \in \Sigma^*.$$

If this holds, we write

$$\mathcal{W}_1 \subseteq \mathcal{W}_2.$$

Equivalently, the problem asks whether

$$\alpha_1 \mu_1(w) \beta_1 \leq \alpha_2 \mu_2(w) \beta_2 \quad \text{for all } w \in \Sigma^*.$$

The containment problem is already highly nontrivial for probabilistic automata, and this complexity carries over to weighted automata. Containment has been studied most extensively for probabilistic automata over the semiring $(\mathbb{R}_{\geq 0}, +, \times)$ and for max-plus/tropical automata over the semiring $(\mathbb{R} \cup \{-\infty\}, \max, +)$. In both settings, the general containment problem is undecidable [10, 4, 16], while restricted ambiguity classes admit positive results: for probabilistic automata, decidability is known when one input is unambiguous and the other is finitely ambiguous (conditional on Schanuel's conjecture) [3, 4], and for finitely ambiguous max-plus automata the corresponding equivalence/containment questions become decidable [11]. More generally, the weighted-automata literature formulates these questions semiring-by-semiring, so the precise status depends strongly on the underlying semiring [3, 10, 4, 16, 11].

In particular, we will verify that the containment problem remains undecidable for stochastic automata, and provide a new algorithm that can instead approximate the containment problem.

- **Theorem 4.12.** The containment problem for WFAs is undecidable. This remains true even when all weights are non-negative.

Proof sketch. We reduce from the universality problem for probabilistic automata, which is known to be undecidable.

A probabilistic automaton can be viewed as a WFA with non-negative weights such that every word is assigned an acceptance probability $p(w) \in [0, 1]$.

The universality problem asks whether

$$p(w) \geq \frac{1}{2} \quad \text{for all } w \in \Sigma^*.$$

Let $\mathcal{W}$ be a probabilistic automaton computing $p(w)$. Construct a WFA $\mathcal{W}_{1/2}$ satisfying

$$\mathcal{L}(\mathcal{W}_{1/2})(w) = \frac{1}{2} \quad \text{for all } w \in \Sigma^*.$$

Then

$$p(w) \geq \frac{1}{2} \iff \mathcal{L}(\mathcal{W}_{1/2})(w) \leq \mathcal{L}(\mathcal{W})(w)$$

for every word w .

Thus deciding containment for WFAs would decide universality for probabilistic automata, contradicting undecidability. ◀

The previous theorem already shows that containment is undecidable for non-negative WFAs. We now strengthen this result further by restricting attention to stochastic WFAs. However, first we will question what the difficulty of this proof is. After all, did we not already prove this for probabilistic automata?

It turns out that probabilistic automata generalise stochastic WFAs, such that the containment problem for stochastic WFAs is a strict subproblem of the containment problem for probabilistic WFAs. To see this, let

$$\mathcal{W} = (\Sigma, n, \alpha, \mu, \beta)$$

be a stochastic WFA. Recall that stochasticity means

$$\sum_{i=1}^n \alpha_i = 1 \quad \text{and} \quad \sum_{\sigma \in \Sigma} \sum_{j=1}^n \mu(\sigma)_{ij} + \beta_i = 1 \quad \text{for all } i \in \{1, \dots, n\}.$$

That is, at every state, the outgoing transition weights together with the final (termination) weight already form a probability distribution. This is almost the definition of a probabilistic automaton, except that termination is encoded implicitly through the final vector β rather than through an explicit symbol.

We remove this discrepancy by introducing a fresh termination symbol $\dagger \notin \Sigma$ and setting $\Sigma' = \Sigma \cup \{\dagger\}$. Define a probabilistic automaton

$$\mathcal{P} = (\Sigma', n+1, \alpha', \mu', F)$$

as follows. The state space is extended by a single absorbing accepting state q_{acc} , so that $\mathcal{P}$ has $n+1$ states, with α' agreeing with α on the original n states and assigning probability 0 to q_{acc} . For $\sigma \in \Sigma$, set

$$\mu'(\sigma) = \begin{pmatrix} \mu(\sigma) & 0 \\ 0 & 0 \end{pmatrix},$$

i.e. $\mu'(\sigma)$ acts exactly as $\mu(\sigma)$ on the original states and never transitions into or out of q_{acc} . For the termination symbol, set

$$\mu'(\dagger) = \begin{pmatrix} 0 & \beta \\ 0 & 1 \end{pmatrix},$$

so that from state $i \leq n$, reading $\dagger$ moves to q_{acc} with probability β_i (and otherwise the automaton is undefined on $\dagger$ outside of this single use, which we may take to mean it self-loops on q_{acc} with probability 1 and is absorbing everywhere else, ensuring row-stochasticity). Finally, let $F = \{q_{\text{acc}}\}$ be the sole accepting state.

By construction, every row of $\mu'(\sigma)$ for $\sigma \in \Sigma'$ sums to 1, so $\mathcal{P}$ is a well-formed probabilistic automaton. Moreover, for every $w \in \Sigma^*$,

$$\Pr_{\mathcal{P}}[w \dagger] = \alpha \mu(w) \beta = \mathcal{L}(\mathcal{W})(w),$$

while $\mathcal{P}$ assigns probability 0 to any string not of the form $w \dagger$. Hence $\mathcal{P}$ reproduces the weighted language of $\mathcal{W}$ exactly, restricted to words ending in the termination symbol.

► **Theorem 4.13.** The containment problem for stochastic WFAs is undecidable.

Proof. We reduce from containment for probabilistic finite automata (PFAs), which is known to be undecidable [19, 10].

Let

$$\mathcal{A} = (\Sigma, n, \alpha, \mu, \beta)$$

be a PFA. Recall that for every symbol $\sigma \in \Sigma$, the transition matrix $\mu(\sigma)$ is row-stochastic:

$$\sum_{j=1}^n \mu(\sigma)_{ij} = 1 \quad \text{for all } i.$$

Let $k = |\Sigma|$. We construct a stochastic WFA

$$\mathcal{W} = (\Sigma, n, \alpha, \mu', \beta')$$

by defining

$$\mu'(\sigma) = \frac{1}{k} \mu(\sigma) \quad \text{for every } \sigma \in \Sigma.$$

Then for every state i ,

$$\begin{aligned} \sum_{\sigma \in \Sigma} \sum_{j=1}^n \mu'(\sigma)_{ij} &= \sum_{\sigma \in \Sigma} \sum_{j=1}^n \frac{1}{k} \mu(\sigma)_{ij} \\ &= \frac{1}{k} \sum_{\sigma \in \Sigma} 1 \\ &= 1. \end{aligned}$$

Choosing $\beta' = 0$ therefore yields a stochastic WFA.
Now let

$$w = \sigma_1 \cdots \sigma_m.$$

Then

$$\begin{aligned} \mathcal{L}(\mathcal{W})(w) &= \alpha \mu'(w) \beta \\ &= \alpha \left(\prod_{i=1}^m \frac{1}{k} \mu(\sigma_i) \right) \beta \\ &= \frac{1}{k^m} \alpha \mu(w) \beta. \end{aligned}$$

Hence

$$\mathcal{L}(\mathcal{W})(w) = \frac{1}{k^{|w|}} \mathcal{L}(\mathcal{A})(w).$$

Let $\mathcal{A}_1, \mathcal{A}_2$ be PFAs and let $\mathcal{W}_1, \mathcal{W}_2$ be the corresponding stochastic WFAs. Since the factor $k^{-|w|}$ is strictly positive,

$$\mathcal{L}(\mathcal{A}_1)(w) \leq \mathcal{L}(\mathcal{A}_2)(w)$$

if and only if

$$\mathcal{L}(\mathcal{W}_1)(w) \leq \mathcal{L}(\mathcal{W}_2)(w).$$

Therefore,

$$\mathcal{A}_1 \subseteq \mathcal{A}_2 \iff \mathcal{W}_1 \subseteq \mathcal{W}_2.$$

Decidability of containment for stochastic WFAs would therefore imply decidability of containment for PFAs, contradicting the known undecidability result. ◀

The undecidability results above show that exact containment is intractable in general. Nevertheless, several important special cases remain decidable.

► **Theorem 4.14.** Let $\mathcal{W}_1$ and $\mathcal{W}_2$ be WFAs over $\mathbb{Q}_{>0}$ on the same alphabet. Assume that $\mathcal{W}_1$ is unambiguous and $\mathcal{W}_2$ is finitely ambiguous. Then the containment problem

$$\mathcal{W}_1 \subseteq \mathcal{W}_2$$

is decidable.

Proof. This is a special case of the decidability results for containment of weighted automata over the probabilistic and max-plus semirings established by Daviaud [3]. ◀

Theorem 4.12, Theorem 4.13, and Theorem 4.14 paint a fairly bleak picture: exact containment is undecidable already for non-negative WFAs, and remains undecidable even when both automata are stochastic, with decidability recovered only under restrictive ambiguity assumptions that need not hold for the WFAs arising from a WCI instance. If containment is to be of any use for verifying or synthesising improvisers in practice, we must therefore be willing to settle for less than an exact yes-or-no answer. In the next section, we show that this is possible whenever the smaller of the two automata has finite total weight, a condition that, as we will see in Chapter 5, is automatically satisfied in the settings we care about.

4.4 Approximating the Containment Problem

Even though exact containment is undecidable in general, one can still obtain approximate guarantees for non-negative WFAs with finite total weight.

► **Theorem 4.15** (Approximate containment). Let

$$\mathcal{W}_A = (\Sigma, n_A, \alpha_A, \mu_A, \beta_A)$$

and

$$\mathcal{W}_B = (\Sigma, n_B, \alpha_B, \mu_B, \beta_B)$$

be non-negative WFAs.

Assume that $\mathcal{W}_A$ has finite total weight:

$$\sum_{w \in \Sigma^*} \mathcal{L}(\mathcal{W}_A)(w) < \infty.$$

Then for every $\varepsilon > 0$ there exists an algorithm that either

- finds a word w such that

$$\mathcal{L}(\mathcal{W}_A)(w) > \mathcal{L}(\mathcal{W}_B)(w),$$

or

- certifies that

$$\mathcal{L}(\mathcal{W}_A)(w) \leq \mathcal{L}(\mathcal{W}_B)(w) + \varepsilon \quad \text{for all } w \in \Sigma^*.$$

The key idea is that finite total weight implies exponential decay of sufficiently long continuations. This allows us to prune large parts of the search space.

► **Algorithm 4.16** (BFS-based ε -approximate containment). **Input:** Non-negative WFAs $\mathcal{W}_A, \mathcal{W}_B$ over the same alphabet Σ , and a tolerance $\varepsilon > 0$, where $\mathcal{W}_A$ has finite total weight.

Output: Either

- a *counterexample word* $w \in \Sigma^*$ such that $\mathcal{L}(\mathcal{W}_A)(w) > \mathcal{L}(\mathcal{W}_B)(w)$,
or
- a *certificate* that $\mathcal{L}(\mathcal{W}_A)(w) \leq \mathcal{L}(\mathcal{W}_B)(w) + \varepsilon$ for every word $w \in \Sigma^*$.

Precomputation. Let

$$M_A = \sum_{\sigma \in \Sigma} \mu_A(\sigma).$$

Since $\mathcal{W}_A$ has finite total weight, $\rho(M_A) < 1$, so

$$S = (I - M_A)^{-1}$$

exists. Intuitively, for a running vector v obtained after reading some prefix of $\mathcal{W}_A$, the quantity $vS\beta_A$ bounds the total weight $\mathcal{W}_A$ can still assign to any continuation of that prefix; this bound is what allows us to prune the search below.

Search. Initialize a queue Q containing only the empty word ε , and repeat the following until Q is empty:

1. Remove a word w from Q .
2. Compute the two word-weights

$$a = \mathcal{L}(\mathcal{W}_A)(w), \quad b = \mathcal{L}(\mathcal{W}_B)(w).$$

3. **(Counterexample check.)** If $a > b$, return w and terminate: this word witnesses that $\mathcal{W}_A \not\subseteq \mathcal{W}_B$ even without any tolerance.
4. Compute the running vector reached by $\mathcal{W}_A$ after reading w ,

$$v = \alpha_A \mu_A(w),$$

and use it to bound the total weight $\mathcal{W}_A$ can assign to *any* extension of w :

$$U(w) = vS\beta_A.$$

5. **(Pruning check.)** If $U(w) \leq \varepsilon$, discard w and continue: no extension of w can violate the ε -tolerant containment, so this branch of the search tree need not be explored further.
6. Otherwise, enqueue every one-symbol extension $w\sigma$ for $\sigma \in \Sigma$, and continue.

Termination. If the queue empties without a counterexample having been found, output the certificate that

$$\mathcal{L}(\mathcal{W}_A)(w) \leq \mathcal{L}(\mathcal{W}_B)(w) + \varepsilon \quad \text{for all } w \in \Sigma^*.$$

► **Theorem 4.17.** The BFS algorithm above is correct and terminates.

Proof. Let

$$v = \alpha_A \mu_A(w).$$

The total weight of all continuations of w equals

$$\sum_{x \in \Sigma^*} \mathcal{L}(\mathcal{W}_A)(wx) = v(I - M_A)^{-1} \beta_A.$$

Since all weights are non-negative,

$$\mathcal{L}(\mathcal{W}_A)(wx) \leq v(I - M_A)^{-1} \beta_A = U(w)$$

for every continuation x .

Hence if

$$U(w) \leq \varepsilon,$$

then every extension of w has weight at most ε , and pruning is safe.

If there exists a word satisfying

$$\mathcal{L}(\mathcal{W}_A)(w) > \mathcal{L}(\mathcal{W}_B)(w),$$

then the BFS eventually explores this word and returns it.

Finally, since $\mathcal{W}_A$ has finite total weight, we know that

$$\rho(M_A) < 1.$$

Therefore

$$\alpha_A M_A^k \rightarrow 0 \quad \text{as } k \rightarrow \infty,$$

implying

$$U(w) \rightarrow 0 \quad \text{as } |w| \rightarrow \infty.$$

Consequently, there exists a depth beyond which every branch is pruned. Only finitely many words are explored, so the algorithm terminates. ◀

The approximate containment procedure above is particularly useful in applications such as control improvisation, where exact guarantees are often unnecessary and additive approximation bounds suffice in practice. For a fixed automaton, the approximation depth grows only logarithmically in $1/\varepsilon$, so the BFS search tree has polynomial size in $1/\varepsilon$.

► **Theorem 4.18** (Complexity of the approximation algorithm). Fix a non-negative WFA $\mathcal{W}_A$ with finite total weight, and fix the alphabet Σ . Then the BFS-based ε -approximation algorithm terminates in time polynomial in $1/\varepsilon$.

Proof. Since $\mathcal{W}_A$ has finite total weight, the aggregate transition matrix

$$M_A = \sum_{\sigma \in \Sigma} \mu_A(\sigma)$$

satisfies $\rho(M_A) < 1$. Because $\mathcal{W}_A$ is fixed, the quantities controlling the decay rate are fixed as well. In particular, there exists a constant $C > 0$ and a constant $\rho \in (0, 1)$ such that every continuation of length k has total weight at most

$$C\rho^k.$$

Hence the algorithm prunes all branches once

$$C\rho^k \leq \varepsilon.$$

Solving for k gives

$$k \geq \frac{\log(C/\varepsilon)}{-\log \rho},$$

so the maximum explored depth is $O(\log(1/\varepsilon))$.

At depth k , the BFS explores at most $|\Sigma|^k$ nodes. Therefore the total number of explored nodes is at most

$$\sum_{i=0}^{O(\log(1/\varepsilon))} |\Sigma|^i = O\left(|\Sigma|^{O(\log(1/\varepsilon))}\right) = O((1/\varepsilon)^c)$$

for some constant $c > 0$ depending only on $\mathcal{W}_A$ and Σ .

Since all matrix-vector computations performed at each node have fixed cost for a fixed automaton, the total running time is polynomial in $1/\varepsilon$. ◀

With this chapter, we have assembled the algorithmic toolkit needed to reason about weighted finite automata as objects in their own right: computing total weight (Section 4.1), sampling from a non-stochastic WFA (Section 4.2), and deciding, or at least approximating, containment between two WFAs (Sections 4.3 and 4.4). Each of these results was phrased in terms of WFAs alone, without reference to control improvisation. We are now in a position to put them to use. In the next chapter, we return to the weighted control improvisation problem of Chapter 3 and show how verification and synthesis of a WCI solution reduce, piece by piece, to exactly the problems studied here.

5 Solving the WCI Problem

In this chapter, we study the two main problems for WCI, that is, the verification and the synthesis problem. The verification problem is to verify if a candidate solution for a WCI problem is valid. The synthesis problem is to give a WCI solution, given a WCI problem. We start by tackling the simpler verification problem.

5.1 Verification

The first computational problem is to determine whether a given automaton is a valid solution to a WCI instance.

- **Problem 5.1** (Verification). Given a WCI problem (W_L, W_U, S) and a WFA I , determine whether I is a WCI solution.

Verification reduces to checking the three conditions of Definition 3.1, one at a time.

Checking the stochastic condition. We must verify that $\text{weight}(I) = 1$. By Theorem 4.8, the total weight of a WFA can be decided and computed in polynomial time whenever it is finite. Since I is given as a WFA over a fixed alphabet, this condition can therefore be checked efficiently.

Checking the soft constraint. The value $\text{weight}(I \odot S)$ can also be computed in polynomial time. The product automaton $I \odot S$ has size polynomial in the sizes of I and S , and its total weight can again be computed using Theorem 4.8. Testing

$$\text{weight}(I \odot S) \geq 1$$

is then a single numerical comparison, exactly as required by Definition 3.1.

Checking the bounds. The remaining condition is the containment requirement

$$\mathcal{L}(W_L) \leq \mathcal{L}(I) \leq \mathcal{L}(W_U).$$

This corresponds exactly to the containment problem studied in Section 4.3. In general, containment is undecidable, and becomes decidable only under additional structural assumptions such as unambiguity or finite ambiguity. These assumptions are restrictive and cannot be expected to hold for arbitrary WCI instances. However, we have also seen that an *approximate* variant of containment is always decidable, provided the smaller of the two automata being compared has finite total weight.

This is precisely the situation we are in. Since I is required to be stochastic, it has total weight exactly 1. For the lower bound, $W_L \leq I$ pointwise forces $\text{weight}(W_L) \leq \text{weight}(I) = 1$, so W_L automatically has finite total weight; if it did not, the containment $\mathcal{L}(W_L) \leq \mathcal{L}(I)$ would fail outright, and verification could reject immediately without needing the approximation algorithm at all. Symmetrically, for the upper bound we are checking $\mathcal{L}(I) \leq \mathcal{L}(W_U)$, where I itself is the smaller automaton and already has finite (indeed, unit) total weight by the stochastic condition. In both directions, then, the automaton on the left-hand side of the inequality has finite total weight, which is exactly the hypothesis required by the approximate containment. We may therefore always apply the BFS-based approximation algorithm to check the bounds, up to a chosen tolerance $\varepsilon > 0$.

Verification therefore reduces to three checks: the stochastic condition and the soft constraint are decidable, and computable, in polynomial time via Theorem 4.8, while the hard bounds reduce to containment, which is undecidable in general but always admits an ε -approximate answer here, since a WCI solution I is stochastic and hence guarantees the finite-weight hypothesis required by the approximation algorithm. In this sense, verification of a WCI solution is practically solved: every part of it is either exactly decidable in polynomial time, or decidable up to an arbitrarily small, user-chosen tolerance.

5.2 Synthesis

The second problem asks whether any valid improviser exists.

- **Problem 5.2** (Synthesis). Given a WCI problem (W_L, W_U, S) , give a WCI solution I .

We can look at this problem at three difficulty levels. First, we can ask whether a (non-constructive) solution exists at all. Then, we can try to (constructively) find a solution. Finally, we can ask how we can find smaller solutions.

We will study different cases of the synthesis problem. First, we look at the most general form, imposing no restrictions whatsoever, and show that this is already undecidable. Having identified the soft constraint as the source of this undecidability, we then develop a strategy for the remaining, better-behaved instances. We first eliminate the lower bound from consideration entirely, reducing every instance to a single residual automaton and two numerical targets. We then dispose of the case where the soft constraint is already satisfied, showing that the resulting purely hard-constrained fragment is always solvable. Next, we turn to the case where the soft constraint still needs to be actively pursued, and identify a simple but fundamental sense in which committing greedily to a single, sufficiently good word is always a safe move. Finally, we show that for a natural and practically relevant class of soft constraints, this greedy idea can be turned into a concrete, terminating synthesis algorithm.

5.2.1 General Synthesis

We begin with the most general form of the problem, imposing no restrictions on the hard or soft constraint automata whatsoever. We show that even the weakest of the three difficulty levels described above, mere existence of a solution, without any requirement to construct one, is already undecidable. The proof proceeds by reduction from the threshold problem for probabilistic automata, which we first re-derive as a special case of the containment problem for WFAs studied in Section 4.3.

- **Problem 5.3** (Threshold problem). Given a WFA $\mathcal{A} = (\Sigma, n, \alpha, \mu, \beta)$ and a threshold $\lambda \in \mathbb{R}$, decide whether there exists a word $w \in \Sigma^*$ such that $\mathcal{L}(\mathcal{A})(w) \geq \lambda$.

It is easy to see that this problem is undecidable, by reducing the containment problem. Given two WFAs $\mathcal{W}_1$ and $\mathcal{W}_2$, we can reduce the problem $\mathcal{W}_1 \leq \mathcal{W}_2$ to the threshold problem for $\mathcal{W}_1 - \mathcal{W}_2$ for $\lambda = 0$.

► **Theorem 5.4.** The general synthesis (existence of a solution) problem for weighted control improvisation is undecidable.

Proof. We reduce from the threshold problem (Problem 5.3).

Let $\mathcal{A} = (\Sigma, n, \alpha, \mu, \beta)$ be a non-negative WFA and let $\lambda \in \mathbb{Q}_{>0}$ be a threshold. We construct, in polynomial time, a WCI instance (W_L, W_U, S) over Σ as follows.

- Let W_L be the zero automaton, assigning weight 0 to every word.
- Let W_U be the unit automaton, assigning weight 1 to every word.
- Let $S := \frac{1}{\lambda} \cdot \mathcal{A}$, i.e. the scalar multiple of $\mathcal{A}$ by $1/\lambda$ (Definition 2.13). This is well defined since $\lambda > 0$, and by Proposition 2.14 $\mathcal{L}(S)(w) = \mathcal{A}(w)/\lambda$ for every $w \in \Sigma^*$.

Since $\mathcal{A}$ is non-negative and $\lambda > 0$, S is non-negative. For every stochastic WFA I we have $0 \leq \mathcal{L}(I)(w) \leq \text{weight}(I) = 1$ for all w , so the hard bounds $W_L \leq \mathcal{L}(I) \leq W_U$ are satisfied automatically. Hence the constructed instance has a solution if and only if there exists a stochastic WFA I with

$$\text{weight}(I \odot S) \geq 1,$$

exactly as required by Definition 3.1. By linearity of the product construction (Proposition 2.12) and scalar multiplication,

$$\begin{aligned} & \text{weight}(I \odot S) \\ &= \sum_{w \in \Sigma^*} \mathcal{L}(I)(w) \mathcal{L}(S)(w) \\ &= \frac{1}{\lambda} \sum_{w \in \Sigma^*} \mathcal{L}(I)(w) \mathcal{A}(w) \\ &= \frac{1}{\lambda} \text{weight}(I \odot \mathcal{A}), \end{aligned}$$

so the solvability condition $\text{weight}(I \odot S) \geq 1$ is equivalent to

$$\text{weight}(I \odot \mathcal{A}) \geq \lambda. \quad (*)$$

Claim. A stochastic WFA I satisfying $(*)$ exists if and only if some word $w \in \Sigma^*$ satisfies $\mathcal{A}(w) \geq \lambda$.

($\Leftarrow$) Suppose some word w^* satisfies $\mathcal{A}(w^*) \geq \lambda$. Let $I := [w^*]$ be the atomic automaton concentrated on w^* with $c = 1$ (Definition 2.15); this I is stochastic, since it has total weight 1 and is non-negative. Then

$$\text{weight}(I \odot \mathcal{A}) = \sum_{w \in \Sigma^*} \mathcal{L}(I)(w) \mathcal{A}(w) = \mathcal{A}(w^*) \geq \lambda,$$

so $(*)$ holds and I is a WCI solution to (W_L, W_U, S) .

$(\Rightarrow)$ Suppose a stochastic WFA I satisfies $(*)$. Since $\mathcal{L}(I)$ is a probability distribution, $\sum_w \mathcal{L}(I)(w) = 1$, so

$$\sum_{w \in \Sigma^*} \mathcal{L}(I)(w)(\mathcal{A}(w) - \lambda) = \text{weight}(I \odot \mathcal{A}) - \lambda \stackrel{(*)}{\geq} 0.$$

If every word satisfied $\mathcal{A}(w) < \lambda$, then every term $\mathcal{L}(I)(w)(\mathcal{A}(w) - \lambda)$ would be at most 0, and strictly negative whenever $\mathcal{L}(I)(w) > 0$; since some word has $\mathcal{L}(I)(w) > 0$ (as I has total weight 1), the sum would be strictly negative, a contradiction. Hence some word w satisfies $\mathcal{A}(w) \geq \lambda$, proving the claim.

Combining both directions with the equivalence between $(*)$ and $\text{weight}(I \odot S) \geq 1$, the constructed WCI instance has a solution if and only if $(\mathcal{A}, \lambda)$ is a yes-instance of the threshold problem. Since the threshold problem is undecidable, the general WCI synthesis problem is undecidable. $\blacktriangleleft$

We see that the undecidability arises entirely from the soft constraint: in our reduction, the hard bounds were trivial, so *every* stochastic automaton already satisfied them, and the entire difficulty was concentrated in the single inequality $\text{weight}(I \odot S) \geq 1$. This suggests a natural way to make progress. Rather than attacking the full problem at once, we first strip away the bookkeeping contributed by the hard bounds, so that only the soft constraint remains to be dealt with. Once this reduction is in place, we turn to the soft constraint itself, and identify a simple but useful sense in which committing to a single, sufficiently good word is always a safe move.

5.2.2 Eliminating the Lower Bound

We now show that, regardless of the soft constraint, every WCI instance can be brought into a normal form in which the lower bound W_L plays no further role, and only a single residual automaton together with two numerical targets remains to be dealt with.

Throughout this section, fix a WCI instance (W_L, W_U, S) with W_L and W_U non-negative and $W_L \leq W_U$ pointwise, and define the residual automaton

$$D := W_U - W_L,$$

which is non-negative by assumption. Every candidate improviser I satisfying $W_L \leq \mathcal{L}(I) \leq W_U$ can then be written uniquely as $I = W_L + J$, for a non-negative weighted language J with $0 \leq J \leq D$ pointwise; conversely, every such J yields a valid I . So bounding I between W_L and W_U is exactly the same thing as bounding J between 0 and D .

Since I is required to be stochastic, we further need $\text{weight}(I) = 1$, i.e.

$$\text{weight}(J) = 1 - \text{weight}(W_L) =: m.$$

A necessary condition for a solution to exist at all is therefore $\text{weight}(W_L) \leq 1 \leq \text{weight}(W_U)$ (otherwise m would be negative, or no non-negative $J \leq D$ with the required weight could exist). This condition also forces $\text{weight}(W_L)$ to be finite, so it can be checked (approximately) in polynomial time by Theorem 4.8. We assume from here on that it holds, so that $m \geq 0$ is well defined.

The soft constraint decomposes along exactly the same split. By linearity of the product construction (Proposition 2.12),

$$\text{weight}(I \odot S) = \text{weight}(W_L \odot S) + \text{weight}(J \odot S),$$

so the requirement $\text{weight}(I \odot S) \geq 1$ becomes

$$\text{weight}(J \odot S) \geq 1 - \text{weight}(W_L \odot S) =: \delta.$$

Putting both observations together, the original WCI instance has a solution if and only if the following residual problem has a solution.

► **Problem 5.5** (Residual synthesis). Given a non-negative WFA D and targets $m \geq 0$ and $\delta \in \mathbb{R}$, find a non-negative weighted language J such that

$$0 \leq J \leq D, \quad \text{weight}(J) = m, \quad \text{weight}(J \odot S) \geq \delta.$$

One degenerate case can be disposed of immediately: if $m = 0$, then $J \equiv 0$ is forced, so the instance is solvable only if $\delta \leq 0$. The remaining question, then, is twofold. First, when $\delta \leq 0$, the soft constraint is already met by any J satisfying only the first two conditions of Problem 5.5; we show in the next subsection that such a J can always be constructed explicitly whenever it exists at all. Second, when $\delta > 0$, the soft constraint must be actively pursued, and it is this harder case that occupies the remainder of the chapter.

We have thus reduced the two hard bounds W_L and W_U , together with the stochastic condition, to a single residual automaton D and two numbers m and δ , which together record how much probability mass is still to be placed, and how much expected score that mass still needs to earn. All that remains is to decide, and ideally construct, a solution to the residual problem. We turn to this question next.

5.2.3 Synthesis without Soft Constraints

We now settle the case $\delta \leq 0$ left open at the end of the previous subsection, in which only the mass constraint

$$0 \leq J \leq D, \quad \text{weight}(J) = m$$

must be satisfied, with no further requirement on $\text{weight}(J \odot S)$. This is exactly the hard-constraint fragment of the WCI instance: a solution here corresponds to a stochastic $I = W_L + J$ respecting the bounds $W_L \leq \mathcal{L}(I) \leq W_U$, with the soft constraint discharged for free. We show that such a J can always be constructed explicitly whenever $m \leq \text{weight}(D)$ (and that no such J exists otherwise), and that the construction depends on whether D itself has finite total weight.

Case 1: The residual automaton has finite total weight

Suppose D has finite total weight, i.e. $\rho(M_D) < 1$ for $M_D = \sum_{\sigma} \mu_D(\sigma)$, so that $\text{weight}(D) < \infty$ by Theorem 4.8. Consider the family of automata

$$J_t := t \cdot D, \quad t \in [0, 1].$$

Each J_t is non-negative and satisfies $0 \leq J_t(w) \leq D(w)$ for every word w , since $0 \leq t \leq 1$. Its total weight,

$$\text{weight}(J_t) = t \cdot \text{weight}(D),$$

is affine in t , ranging from 0 at $t = 0$ to $\text{weight}(D)$ at $t = 1$. Provided $m \leq \text{weight}(D)$, the value

$$t^* = \frac{m}{\text{weight}(D)}$$

(taking $t^* = 0$ if $\text{weight}(D) = 0$, in which case $m = 0$ already) satisfies $t^* \in [0, 1]$ and $\text{weight}(J_{t^*}) = m$. Since $\text{weight}(D)$ is computable in polynomial time by Theorem 4.8, t^* is obtained directly, without search. Setting $J := J_{t^*}$ gives a non-negative weighted language with $0 \leq J \leq D$ and $\text{weight}(J) = m$, so $I := W_L + J$ is a non-negative WFA with $\text{weight}(I) = 1$ and $W_L \leq \mathcal{L}(I) \leq W_U$; by Theorem 4.9, I is equivalent to a stochastic WFA of the same size, solving the hard-constraint fragment of the WCI instance. Conversely, if $m > \text{weight}(D)$, no non-negative $J \leq D$ can reach total weight m , so the instance is infeasible; this can also be checked in polynomial time.

Case 2: The residual automaton has infinite total weight

Now suppose $\rho(M_D) \geq 1$, so $\text{weight}(D)$ diverges. The construction of Case 1 no longer applies, since $t \mapsto \text{weight}(J_t)$ jumps from 0 to

∞ for every $t > 0$. Instead of scaling D uniformly by a scalar, we scale it non-uniformly by word length, damping longer words more aggressively.

Let $\rho_D := \rho(M_D) \geq 1$. For $\varepsilon \in (0, 1/\rho_D)$, define the ε -damped automaton

$$D_\varepsilon := (\Sigma, n_D, \alpha_D, \varepsilon \cdot \mu_D, \beta_D),$$

obtained from D by scaling every transition matrix by ε . A path labelled by a word w of length n picks up a factor ε^n from its n transitions, so

$$\mathcal{L}(D_\varepsilon)(w) = \varepsilon^{|w|} \cdot \mathcal{L}(D)(w)$$

for every $w \in \Sigma^*$, and hence $0 \leq D_\varepsilon(w) \leq D(w)$ pointwise, as required of a candidate $J \leq D$.

The aggregate matrix of D_ε is εM_D , with spectral radius $\varepsilon \rho_D < 1$, so D_ε has finite total weight by Theorem 4.8, with

$$\text{weight}(D_\varepsilon) = \alpha_D (I - \varepsilon M_D)^{-1} \beta_D.$$

As a function of ε on $(0, 1/\rho_D)$, this quantity is continuous and monotonically non-decreasing, since each coefficient $\varepsilon^n D(w)$ is non-decreasing in $\varepsilon \geq 0$. It tends to $\alpha_D \beta_D$ as $\varepsilon \rightarrow 0^+$ and to ∞ as $\varepsilon \rightarrow (1/\rho_D)^-$. Since $\text{weight}(D) = \infty$, the target value m lies in this range, so by the Intermediate Value Theorem there exists $\varepsilon^* \in (0, 1/\rho_D)$ with

$$\text{weight}(D_{\varepsilon^*}) = m.$$

Unlike Case 1, $\varepsilon \mapsto \text{weight}(D_\varepsilon)$ is not affine, so ε^* cannot in general be obtained in closed form. Since $\text{weight}(D_\varepsilon)$ is continuous, monotonically non-decreasing, and computable in polynomial time for fixed ε by Theorem 4.8, a bisection search over $(0, 1/\rho_D)$ converges to ε^* to any desired precision δ_0 in $O(\log(1/\delta_0))$ evaluations; any comparable numerical root-finder for a monotone continuous function applies equally well. Setting $J := D_{\varepsilon^*}$ gives a non-negative weighted language with $0 \leq J \leq D$ pointwise and $\text{weight}(J) = m$, so $I := W_L + J$ again solves the hard-constraint fragment of the WCI instance, by the same argument as in Case 1.

We have now seen that, whenever the soft constraint is already satisfied, the residual problem is always solvable in an explicit and computationally tractable sense: whenever $m \leq \text{weight}(D)$, Case 1 and Case 2 together show how to construct a suitable J , either in closed form or via bisection, regardless of whether D itself has finite total weight, and no such J exists when $m > \text{weight}(D)$. This construction, however, makes no attempt to account for the soft constraint; the mass m was distributed across D in whatever way happened to be convenient, be it uniformly via a scalar t or via length-based damping. To solve the remaining case $\delta > 0$, we must instead distribute this mass deliberately, favouring words on which the soft-constraint automaton S scores highly. We develop the key idea behind such a deliberate distribution next.

5.2.4 A Greedy Approach

Having settled the case $\delta \leq 0$, we now investigate the remaining, harder case $\delta > 0$, in which the soft-constraint automaton S must be used to decide whether a suitable J exists at all. The general problem for arbitrary S is undecidable: Theorem 5.4 already used W_L trivial and W_U the unit automaton, so that D and m were fixed and only δ and S carried the difficulty. Rather than attempting to decide the residual problem outright, we isolate a single structural observation on which any algorithm for it would have to rely: committing as much probability mass as possible to a single word whose score already exceeds what is required on average is always a safe move, in the sense that doing so never turns a solvable instance into an unsolvable one.

Assume throughout this section that $m > 0$ and $\delta > 0$, the only case left open by Section 5.2.3. Define

$$v := \frac{\delta}{m},$$

the minimal average score that the mass m must achieve; a word w with $S(w) > v$ scores strictly better than this required average.

► **Theorem 5.6** (Greedy safety). Let D be a non-negative WFA, let $m > 0$ and $\delta > 0$, and let w be a word with $D(w) > 0$ and $S(w) > v = \delta/m$. Set

$$\alpha := \min\{D(w), m\},$$

and define the reduced instance

$$D' := D - \alpha[w], \quad m' := m - \alpha, \quad \delta' := \delta - \alpha S(w).$$

Then Problem 5.5 is solvable for (D, m, δ) if and only if it is solvable for (D', m', δ') .

Proof. ($\Leftarrow$) Suppose J' solves the reduced instance, i.e. $0 \leq J' \leq D'$, $\text{weight}(J') = m'$, and $\text{weight}(J' \odot S) \geq \delta'$. Set $J := J' + \alpha[w]$. Since $J'(w) \leq D'(w) = D(w) - \alpha$, we have $J(w) = J'(w) + \alpha \leq D(w)$, and $J(v) = J'(v) \leq D'(v) = D(v)$ for $v \neq w$, so $0 \leq J \leq D$. Moreover $\text{weight}(J) = \text{weight}(J') + \alpha = m' + \alpha = m$, and

$$\text{weight}(J \odot S) = \text{weight}(J' \odot S) + \alpha S(w) \geq \delta' + \alpha S(w) = \delta,$$

so J solves the original instance.

($\Rightarrow$) Suppose (D, m, δ) is solvable. We first show that it is solvable by some J with $J(w) \geq \alpha$; the reduced instance is then solved by $J - \alpha[w]$, exactly as in the argument above run in reverse: writing $J^- := J - \alpha[w]$, we get $J^-(w) = J(w) - \alpha \geq 0$ and $J^-(w) \leq D(w) - \alpha = D'(w)$, while

$J^-(v) = J(v) \leq D(v) = D'(v)$ for $v \neq w$, so $0 \leq J^- \leq D'$; also $\text{weight}(J^-) = m - \alpha = m'$ and $\text{weight}(J^- \odot S) = \text{weight}(J \odot S) - \alpha S(w) \geq \delta - \alpha S(w) = \delta'$.

It remains to find some solution J with $J(w) \geq \alpha$. Let J be any solution to (D, m, δ) , and write $x := J(w)$. If $x \geq \alpha$ we are already done. Otherwise $x < \alpha \leq m$, so $m - x > 0$. Let R denote J restricted to words other than w , so that $R(w) = 0$, $R(v) = J(v)$ for $v \neq w$, and $\text{weight}(R) = m - x$. Set

$$c := \frac{m - \alpha}{m - x} \in [0, 1),$$

which is well defined and lies in $[0, 1)$ since $0 \leq m - \alpha < m - x$, and define

$$J'(w) := \alpha, \quad J'(v) := c \cdot R(v) \quad (v \neq w).$$

Since $c < 1$, we have $0 \leq J'(v) \leq R(v) \leq D(v)$ for $v \neq w$, and $J'(w) = \alpha \leq D(w)$, so $0 \leq J' \leq D$. Its total weight is

$$\text{weight}(J') = \alpha + c \cdot \text{weight}(R) = \alpha + c(m - x) = \alpha + (m - \alpha) = m,$$

as required. For the soft constraint, write $\varepsilon := \alpha - x > 0$; since $\text{weight}(R \odot S) = \text{weight}(J \odot S) - xS(w) \geq \delta - xS(w)$, we get

$$\text{weight}(J' \odot S) = \alpha S(w) + c \cdot \text{weight}(R \odot S) \geq \alpha S(w) + c(\delta - xS(w)).$$

A direct computation, using $c = (m - \alpha)/(m - x)$, gives

$$\alpha S(w) + c(\delta - xS(w)) - \delta = \frac{\varepsilon m}{m - x}(S(w) - \nu),$$

which is strictly positive since $\varepsilon, m, m - x > 0$ and $S(w) > \nu$ by assumption. Hence $\text{weight}(J' \odot S) \geq \delta$, and J' is a solution to (D, m, δ) with $J'(w) = \alpha$, as required. $\blacktriangleleft$

Theorem 5.6 says that greed is never premature: as soon as some available word scores above the required average, we may allocate it as much mass as the residual automaton and the remaining budget allow, and any solution that existed before is still reachable afterwards, just phrased in terms of the smaller instance (D', m', δ') . Repeating this step, each time on whatever residual problem remains, only ever shrinks the mass still to be placed and the deficit still to be cleared, while never discarding a solution that was there to begin with.

This is exactly the mechanism one would want at the heart of a synthesis algorithm for Problem 5.5: search for a sufficiently good word, commit to it fully, and recurse on what is left. Turning this into a terminating algorithm, however, requires more than Theorem 5.6 alone. One still needs a way of actually finding such a word w in the first place, for instance an oracle, or a decidable search over some effectively

enumerable class of candidates, together with a guarantee that the reduced instances eventually reach $\delta' \leq 0$ or exhaust D , rather than continuing forever. In the next section, we identify a natural class of soft-constraint automata for which exactly such a search is effective, and for which the greedy idea above can be turned into a concrete, terminating algorithm without any oracle at all.

5.2.5 Finitely Insufficient Soft Constraints

We now identify a natural class of soft-constraint automata for which synthesis is not merely approximable or oracle-dependent, but decidable outright, with a procedure that always terminates and never requires an external oracle. The key property we will exploit is that a soft-constraint automaton often assigns low scores to only finitely many words, and that this finiteness can be checked at the single, fixed threshold 1, exactly the threshold appearing in Definition 3.1 itself, rather than at an arbitrary value chosen up front. We formalise this property as constructive finite insufficiency, show that both scoring automata from the running examples of Chapter 3 satisfy it at every positive scale, and give an exact synthesis procedure for this class, together with a matching proof that it never reports infeasibility incorrectly.

► **Definition 5.7** (Constructively finitely insufficient WFA). A non-negative WFA S is *constructively finitely insufficient* if the language

$$\{w \in \Sigma^* : S(w) < 1\}$$

is finite, and can moreover be computed explicitly.

Three remarks are in order. First, on effectiveness: the procedure below will need to know that $\{w : S(w) < c\}$ is finite and computable for values c other than 1. This causes no difficulty, since for any $c > 0$,

$$\{w \in \Sigma^* : S(w) < c\} = \{w \in \Sigma^* : (c^{-1} \cdot S)(w) < 1\},$$

using scalar multiplication of WFAs (Definition 2.13). Checking finiteness at c is therefore literally the same check as in Definition 5.7, applied to the rescaled automaton $c^{-1} \cdot S$ instead of to S itself; the threshold used is always 1, only the automaton being tested changes. For this to be available whenever the procedure needs it, we will assume that S has the stronger property that $c^{-1} \cdot S$ is constructively finitely insufficient for every $c > 0$. This holds for many soft-constraint automata of practical interest, since such automata typically accumulate a strictly positive reward along every cycle reachable from the initial state, which gives an explicit bound relating $|w|$ to $S(w)$; this bound scales linearly under multiplying S by a positive constant, so a single bounded search over word length, rescaled by c , computes $\{w : S(w) < c\}$ for every c .

Second, on examples: both scoring automata introduced in Chapter 3 have this stronger property. In the railway example, every symbol contributes at least 50/1000 to the score, so $\{w : S(w) < c\}$ is contained in the set of words of length less than $20c$, which is finite and computable for every $c > 0$. In the park-routing example, every edge traversed contributes at least one point towards the total, so the same argument applies. Third, on classical control improvisation: recall that a classical CI instance is translated into a WCI instance by encoding its Boolean soft constraint $S \subseteq \Sigma^*$ as a scoring automaton T with $T(w) = 1/(1 - \epsilon)$ for $w \in S$ and $T(w) = 0$ otherwise, alongside hard bounds W_L, W_U that vanish outside $H \cap \Sigma^{\leq n}$ for the classical hard language H and length bound n . Since $1/(1 - \epsilon) \geq 1$, we have $T(w) < 1$ exactly when $w \notin S$, so $\{w : T(w) < 1\} = \Sigma^* \setminus S$, which is generally infinite; T is not constructively finitely insufficient. However, in the translated instance T is only ever evaluated against a residual automaton D that is itself supported on $H \cap \Sigma^{\leq n}$, a finite language, since the hard bounds already vanish beyond it. The residual problem therefore only ever needs to distinguish finitely many words with nonzero D -weight, which is precisely the situation handled by the finite-support case of Case A and Case B below, so every WCI instance translated from classical control improvisation falls squarely within the scope of the procedure developed in this section.

► **Remark 5.8.** Suppose $c^{-1} \cdot S$ is constructively finitely insufficient for every $c > 0$, as in the two running examples. Then S is unbounded above: if $S(w) \leq M$ for all w , then taking any $c > M$ would make $\{w : S(w) < c\} = \Sigma^*$ infinite, a contradiction. Consequently, for every $\nu \geq 0$ the set $\{w : S(w) > \nu\}$ is infinite (its complement is finite), while $\{w : S(w) \leq \nu\}$ is finite and computable.

Setup. Fix a WCI instance (W_L, W_U, S) with S constructively finitely insufficient at every positive scale (or, as discussed above, with S evaluated only against a finitely supported residual automaton, as happens for instances translated from classical CI), and recall the residual automaton $D := W_U - W_L$, mass $m := 1 - \text{weight}(W_L)$, and deficit $\delta := 1 - \text{weight}(W_L \odot S)$ from Section 5.2.2. We assume the necessary condition $\text{weight}(W_L) \leq 1 \leq \text{weight}(W_U)$ has already been checked, and that $\delta > 0$ and $m > 0$; otherwise the instance is settled by Section 5.2.3. Recall $\nu := \delta/m$ from Section 5.2.4, the minimal average score that the mass m must achieve; since $\delta, m > 0$, we have $\nu > 0$. Set

$$T := (2\nu)^{-1} \cdot S,$$

so that $T(w) < 1$ exactly when $S(w) < 2\nu$. Since T is constructively finitely insufficient, the language $\{w : S(w) < 2\nu\}$ is finite and can be

computed explicitly; this is the only fact about S that the procedure below relies on.

Case A: Enough mass strictly above the threshold. Define

$$\begin{aligned} L &:= \{w \in \Sigma^* : S(w) \leq \nu\}, \\ H &:= \Sigma^* \setminus L = \{w : S(w) > \nu\}. \end{aligned}$$

Since $\nu < 2\nu$, we have $L \subseteq \{w : S(w) < 2\nu\} = \{w : T(w) < 1\}$, so L is finite and computable, being a subset, checkable pointwise, of a finite computable set; H is infinite. Compute $\text{weight}(D)$ (possibly $+\infty$), and the finite sum $c_D := \sum_{w \in L} D(w)$. Set

$$W_H := \text{weight}(D) - c_D,$$

the total weight D assigns to H (possibly $+\infty$).

Suppose $W_H \geq m$. Fix any fixed effective enumeration $w_1, w_2, w_3, \dots$ of H (for instance, by increasing length and then lexicographically). Since D is non-negative, the partial sums $\sum_{i=1}^k D(w_i)$ increase monotonically to W_H regardless of the enumeration order, so there is a least index k with

$$\sum_{i=1}^k D(w_i) \geq m.$$

Let $r := m - \sum_{i=1}^{k-1} D(w_i) \in (0, D(w_k)]$, and define

$$J := \sum_{i=1}^{k-1} D(w_i) [w_i] + r [w_k],$$

a finite sum of atomic automata, hence itself a WFA, with $0 \leq J \leq D$ pointwise and $\text{weight}(J) = m$ exactly. Since every word used lies in H , every term satisfies $S(w_i) > \nu$ strictly, so

$$\begin{aligned} \text{weight}(J \odot S) &= \sum_{i=1}^{k-1} D(w_i) S(w_i) + r S(w_k) \\ &> \nu \sum J(w) = \nu m = \delta. \end{aligned}$$

Return $I := W_L + J$; it is a WCI solution. This construction is in fact exactly what Theorem 5.6 licenses, applied repeatedly to a finite sequence of words from H : each w_i scores strictly above the required average ν , so committing to it is safe, and constructive finite insufficiency of T guarantees that finitely many such commitments already exhaust the required mass m , so the process terminates in finitely many steps without ever needing an oracle. Note that this construction does not need to be optimal; any finite subset of H summing to m already clears the threshold, since *every* word in H individually exceeds the required average.

Case B: Not enough mass above the threshold. Now suppose $W_H < m$ (so W_H is finite). Define

$$J_{\text{top}} := D - \sum_{w \in L} D(w) [w],$$

again a WFA obtained from D by a finite correction, agreeing with D on H and vanishing on L , so $\text{weight}(J_{\text{top}}) = W_H$. Compute $\text{weight}(D \odot S)$. If this diverges, set $T_{\text{top}} := \infty$; in this case $T_{\text{top}} \geq \delta$ automatically and any completion of the remaining mass within L yields a solution, so assume henceforth that $\text{weight}(D \odot S)$ is finite. Compute the finite sum $c_{DS} := \sum_{w \in L} D(w)S(w)$ exactly, and set

$$T_{\text{top}} := \text{weight}(D \odot S) - c_{DS} = \text{weight}(J_{\text{top}} \odot S).$$

Let $m' := m - W_H > 0$ be the remaining mass to allocate, and $\delta' := \delta - T_{\text{top}}$ the remaining score deficit.

If $\delta' \leq 0$: choose any ℓ with $0 \leq \ell \leq D|_L$ pointwise and $\text{weight}(\ell) = m'$. Such an ℓ exists because $\sum_{w \in L} D(w) = \text{weight}(D) - W_H \geq m - W_H = m'$, using $\text{weight}(D) \geq m$. Set $J := J_{\text{top}} + \ell$ and return $I := W_L + J$; then $\text{weight}(J \odot S) = T_{\text{top}} + \text{weight}(\ell \odot S) \geq T_{\text{top}} \geq \delta$.

If $\delta' > 0$: since L is finite, sort it by score in *descending* order, $w'_1, \dots, w'_\ell$ with $S(w'_1) \geq \dots \geq S(w'_\ell)$, and let j be the least index with $\sum_{i=1}^j D(w'_i) \geq m'$ (which exists since $\sum_{w \in L} D(w) \geq m'$, as above). Let $r' := m' - \sum_{i=1}^{j-1} D(w'_i)$, and define

$$\text{best_low} := \sum_{i=1}^{j-1} D(w'_i)S(w'_i) + r' S(w'_j).$$

If $T_{\text{top}} + \text{best_low} \geq \delta$, set

$$\ell := \sum_{i=1}^{j-1} D(w'_i) [w'_i] + r' [w'_j], \quad J := J_{\text{top}} + \ell,$$

and return $I := W_L + J$, a WCI solution with $\text{weight}(J \odot S) = T_{\text{top}} + \text{best_low} \geq \delta$. Otherwise, report **infeasible**.

► **Lemma 5.9.** In the setting of Case B, for every non-negative weighted language J with $0 \leq J \leq D$ pointwise and $\text{weight}(J) = m$,

$$\text{weight}(J \odot S) \leq T_{\text{top}} + \text{best_low}.$$

Proof. Write $J = J_H + J_L$, where $J_H(w) := J(w)$ for $w \in H$ and 0 otherwise, and J_L analogously for L ; since H, L partition Σ^* ,

$$\text{weight}(J \odot S) = \text{weight}(J_H \odot S) + \text{weight}(J_L \odot S).$$

Bound on the H-part. Since $0 \leq J(w) \leq D(w)$ and $S(w) \geq 0$ for every $w \in H$, we have $J(w)S(w) \leq D(w)S(w)$ pointwise, and summing this non-negative termwise inequality gives

$$\text{weight}(J_H \odot S) = \sum_{w \in H} J(w)S(w) \leq \sum_{w \in H} D(w)S(w) = T_{\text{top}}.$$

This bound holds regardless of $\text{weight}(J_H)$.

Reduction to full use of H. Let $x := \text{weight}(J_H)$ and $y := \text{weight}(J_L) = m - x$. Since $J_H \leq D$ pointwise on H , $x \leq W_H$, so $y \geq m - W_H = m'$. If $y = m'$, then $\text{weight}(J_L \odot S) \leq \text{best_low}$ directly, since best_low is by construction the maximum of $\text{weight}(J'_L \odot S)$ over all $0 \leq J'_L \leq D|_L$ with $\text{weight}(J'_L) = m'$ exactly, a consequence of the rearrangement inequality on the finite set L : reallocating mass from a lower-scoring word in L to unused capacity at a higher-scoring word in L never decreases the sum, so sorting L by score and filling greedily, as done above, is optimal for a fixed target mass m' . Combined with the H -bound, this already gives the claim when $y = m'$.

Suppose instead $y > m'$, and let $\varepsilon := y - m' > 0$. Since $W_H - x = W_H - (m - y) = \varepsilon > 0$, there is unused capacity in H totalling exactly ε ; being a sum of non-negative terms, this unused capacity can be found in finitely many words $u_1, \dots, u_p \in H$ with slack $D(u_i) - J(u_i) > 0$ and amounts $\varepsilon_1, \dots, \varepsilon_p > 0$, $\varepsilon_i \leq D(u_i) - J(u_i)$, summing to ε . Likewise, since L is finite and J_L carries total mass $y \geq \varepsilon$, we may withdraw a total of ε from J restricted to L in some valid way. Define J' by adding ε_i to $J(u_i)$ for each i and removing this total ε from J on L . Then J' is non-negative, $0 \leq J' \leq D$ pointwise, $\text{weight}(J') = \text{weight}(J) = m$, and

$$\text{weight}(J'_H) = x + \varepsilon = m - m' = W_H,$$

so J' falls into the case $y' = m'$ already handled. Since every $u_i \in H$ satisfies $S(u_i) > \nu$ strictly, while every unit of mass removed from L carries a score of at most ν ,

$$\begin{aligned} & \text{weight}(J' \odot S) - \text{weight}(J \odot S) \\ &= \sum_i \varepsilon_i S(u_i) - (\text{removed score}) \geq \nu \varepsilon - \nu \varepsilon = 0, \end{aligned}$$

so $\text{weight}(J \odot S) \leq \text{weight}(J' \odot S) \leq T_{\text{top}} + \text{best_low}$, completing the proof. $\blacktriangleleft$

- **Theorem 5.10.** If S is constructively finitely insufficient at every positive scale, the procedure above decides the residual synthesis problem for (W_L, W_U, S) exactly: it returns a WCI solution I whenever one exists, and correctly reports infeasibility otherwise. In particular, WCI synthesis is decidable whenever the soft-constraint automaton has this property.

Proof. In Case A, and in Case B when $\delta' \leq 0$ or when $T_{\text{top}} + \text{best_low} \geq \delta$, the automaton J constructed above satisfies $0 \leq J \leq D$, $\text{weight}(J) = m$, and $\text{weight}(J \odot S) \geq \delta$ exactly, as verified in each case; hence $I := W_L + J$ is a genuine WCI solution by the equivalence established in Section 5.2.2. Conversely, if the procedure reports infeasible, this only occurs in Case B with $\delta' > 0$ and $T_{\text{top}} + \text{best_low} < \delta$; by Lemma 5.9, every valid J satisfies $\text{weight}(J \odot S) \leq T_{\text{top}} + \text{best_low} < \delta$, so no solution exists. ◀

Sections 5.2.2–5.2.5 show that the undecidability of Theorem 5.4 can be circumvented whenever the soft constraint has enough structure to guarantee that a good candidate word is never far away. One might expect the remaining hardness to be tied up entirely with ambiguity: after all, Theorem 4.14 shows that containment, the ingredient underlying the hard-bound side of both verification and synthesis, becomes exactly decidable once the automata involved are unambiguous or finitely ambiguous, and the residual automata D constructed throughout this chapter typically inherit whatever ambiguity the original hard bounds had. It is therefore natural to hope that restricting the entire WCI instance, hard bounds *and* soft constraint alike, to unambiguous or even deterministic WFAs would place synthesis safely inside the tractable fragment identified above. The next section shows that this hope is unfounded: even the most restricted class of WFAs we have studied, deterministic automata with only two states, can already force a synthesis instance outside the reach of any WFA solution whatsoever.

5.3 Synthesis with Deterministic Soft Constraints

The synthesis algorithms of Section 5.2.5 rely, in their exact form, on the soft-constraint automaton being finitely insufficient (Definition 5.7): only finitely many words may score below any given threshold. This section shows that this assumption is not a mere technical convenience, and that the difficulty it guards against is not an artefact of ambiguity either. Deterministic WFAs are, in a precise sense, among the weakest and most restricted automata studied in this thesis: every word has a unique accepting path, minimisation is immediate, and, as just recalled, containment and equivalence between them are exactly decidable rather than merely approximable. It is therefore somewhat surprising that this restricted class is already expressive enough to produce a WCI instance that would admit a valid probability distribution, but this distribution cannot be generated by a WFA.

We construct a WCI instance (W_L, W_U, S) in which every automaton is a two-state, deterministic WFA, in which the lower bound is trivial ($W_L \equiv 0$), and in which the supremum of the achievable soft-constraint value over all non-negative weighted languages respecting the hard bounds is exactly the threshold 1, yet this supremum is attained by a *unique* weighted language, and that language is not the semantics of any WFA. The instance therefore has no solution in the sense of Definition 3.1, even though it can be approximated arbitrarily closely by an explicit, increasingly large sequence of WFA candidates. This shows that the obstruction to synthesis identified in Theorem 5.4 already occurs for the simplest class of WFAs one could hope to reason about.

Fix $\Sigma = \{a, b\}$. Define three two-state WFAs. The soft-constraint automaton $S = (\Sigma, 2, \alpha_S, \mu_S, \beta_S)$ is given by

$$\alpha_S = \begin{pmatrix} c & 0 \end{pmatrix}, \quad \mu_S(a) = \begin{pmatrix} 0 & 2 \\ 2 & 0 \end{pmatrix}, \quad \mu_S(b) = \begin{pmatrix} 0 & \frac{1}{2} \\ \frac{1}{2} & 0 \end{pmatrix}, \quad \beta_S = \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

where $c > 0$ is a constant fixed in Proposition 5.15 below. The upper-bound automaton $W_U = (\Sigma, 2, \alpha_U, \mu_U, \beta_U)$ has the same shape,

$$\alpha_U = \begin{pmatrix} \frac{15}{2} & 0 \end{pmatrix}, \quad \mu_U(a) = \mu_U(b) = \begin{pmatrix} 0 & \frac{1}{8} \\ \frac{1}{8} & 0 \end{pmatrix}, \quad \beta_U = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

The lower bound is the zero automaton, $W_L \equiv 0$, so the hard constraint reduces to $0 \leq \mathcal{L}(I) \leq \mathcal{L}(W_U)$ for a candidate solution I . Both S and W_U are deterministic: from either state, each symbol has a unique successor, so both automata are in particular unambiguous.

Since every transition swaps the two states, a run on a word w ends in state 1 if $|w|$ is even and in state 2 if $|w|$ is odd. As α_S, α_U place all initial weight on state 1 and β_S, β_U place all final weight on state 2, both automata assign weight 0 to every even-length word.

► **Proposition 5.11.** For every $w \in \Sigma^*$,

$$\mathcal{L}(S)(w) = \begin{cases} c \cdot 2^{|w|_a - |w|_b} & |w| \text{ odd}, \\ 0 & |w| \text{ even}, \end{cases}$$

$$\mathcal{L}(W_U)(w) = \begin{cases} \frac{15}{2} \left(\frac{1}{8}\right)^{|w|} & |w| \text{ odd}, \\ 0 & |w| \text{ even}. \end{cases}$$

Proof. Both automata are deterministic, so for each w there is exactly one accepting path, and its weight is the product of the initial weight, the transition weights read off w , and the final weight. Reading w contributes a factor 2 for each a and $\frac{1}{2}$ for each b in S , giving $c \cdot 2^{|w|_a} 2^{-|w|_b} = c \cdot 2^{|w|_a - |w|_b}$ when $|w|$ is odd (so that the path ends in the accepting state), and 0 otherwise. In W_U , both symbols contribute a factor $\frac{1}{8}$ regardless of identity, giving $\frac{15}{2} \left(\frac{1}{8}\right)^{|w|}$ for odd $|w|$. ◀

The essential feature of $\mathcal{L}(W_U)$ is that it depends only on the *length* of w , treating a and b symmetrically, while $\mathcal{L}(S)$ depends only on the *excess* $d(w) := |w|_a - |w|_b$, which is necessarily an odd integer whenever $|w|$ is odd, and takes every odd value.

► **Lemma 5.12.** $\sum_{w : d(w) > 0} \mathcal{L}(W_U)(w) = 1.$

Proof. Fix an odd length n . Since $\mathcal{L}(W_U)$ is constant, equal to $\frac{15}{2} 8^{-n}$, on all 2^n words of length n , the total weight W_U assigns to length- n words is $\frac{15}{2} (2/8)^n = \frac{15}{2} (1/4)^n$. As μ_U treats a and b identically, exactly as many length- n words satisfy $d(w) > 0$ as satisfy $d(w) < 0$; since n is odd there are no words with $d(w) = 0$, so exactly half of the 2^n words have $d(w) > 0$. Hence the W_U -mass on $\{w : |w| = n, d(w) > 0\}$ is $\frac{15}{4} (1/4)^n$. Summing over odd n ,

$$\sum_{n \text{ odd}} \frac{15}{4} \left(\frac{1}{4}\right)^n = \frac{15}{4} \cdot \frac{1/4}{1 - 1/16} = \frac{15}{4} \cdot \frac{4}{15} = 1. \quad \blacktriangleleft$$

Consider the residual synthesis problem for this instance in the sense of Section 5.2.2: since $W_L \equiv 0$, we have $D = W_U$, $m = 1$, and we must find a non-negative weighted language J with $0 \leq J \leq W_U$, $\text{weight}(J) = 1$, maximising $\text{weight}(J \odot S)$ (or, for solvability, reaching a value ≥ 1).

Define G^* by

$$G^*(w) := \begin{cases} \mathcal{L}(W_U)(w) & d(w) > 0, \\ 0 & \text{otherwise.} \end{cases}$$

► **Proposition 5.13.** G^* is the unique non-negative weighted language with $0 \leq G^* \leq \mathcal{L}(W_U)$ pointwise and $\text{weight}(G^*) = 1$, and for every such J ,

$$\text{weight}(J \odot S) \leq \text{weight}(G^* \odot S),$$

with equality only for $J = G^*$.

Proof. By Lemma 5.12, $\text{weight}(G^*) = 1$ and $0 \leq G^* \leq \mathcal{L}(W_U)$, so G^* is admissible. Since $\mathcal{L}(S)(w) = c \cdot 2^{d(w)}$ depends only on $d(w)$ and is strictly increasing in $d(w)$, any admissible J with positive mass on some word v with $d(v) \leq 0$ satisfies $\mathcal{L}(S)(v) \leq c$, while $c \cdot 2^d > c$ for every $d > 0$; by Lemma 5.12 the entire admissible mass budget of 1 can be, and must be, absorbed by words with $d(w) > 0$ if the average score is to reach its supremum, since these words already saturate the whole budget. Precisely: if $J \neq G^*$, then either J assigns positive weight to some v with $d(v) \leq 0$, or $J(w) < \mathcal{L}(W_U)(w)$ for some w with $d(w) > 0$. In the first case, since $\text{weight}(J) = 1 = \sum_{d(w) > 0} \mathcal{L}(W_U)(w)$, admissibility forces $J(w) < \mathcal{L}(W_U)(w)$ for some w with $d(w) > 0$ as well (otherwise J would exceed total mass 1). In either case there is a word w_0 with $d(w_0) > 0$ at which J has strictly unused capacity $\mathcal{L}(W_U)(w_0) - J(w_0) > 0$, and correspondingly positive mass $J(v) > 0$ placed either on a word v with $d(v) \leq 0 < d(w_0)$, or, after redistributing, the shortfall at w_0 is compensated elsewhere with strictly lower excess. Moving mass $\varepsilon > 0$ from v to w_0 (possible while respecting $J \leq W_U$, since w_0 has spare capacity) strictly increases $\text{weight}(J \odot S)$, since $\mathcal{L}(S)(w_0) > \mathcal{L}(S)(v)$. Hence J was not optimal, and G^* is the unique maximiser. ◀

The value $\text{weight}(G^* \odot S)$ can be computed in closed form.

► **Lemma 5.14.** For $0 < P, Q$ with $P + Q < 1$,

$$\sum_{n_a > n_b \geq 0} \binom{n_a + n_b}{n_a} P^{n_a} Q^{n_b} = \frac{1}{\sqrt{1 - 4PQ}} \cdot \frac{P C(PQ)}{1 - P C(PQ)},$$

$$C(z) := \frac{1 - \sqrt{1 - 4z}}{2z},$$

where C is the Catalan generating function.

Proof sketch. Writing $n_a = n_b + d$ for $d \geq 1$, the standard generating-function identity $\sum_{n \geq 0} \binom{2n+d}{n} z^n = C(z)^d / \sqrt{1 - 4z}$ (verified directly from $C(z) = 1 + z C(z)^2$ by induction on d) gives, for fixed d ,

$$\sum_{n_b \geq 0} \binom{2n_b + d}{n_b} P^{n_b+d} Q^{n_b} = P^d \cdot \frac{C(PQ)^d}{\sqrt{1 - 4PQ}}.$$

Summing the resulting geometric series over $d \geq 1$ gives the claim, valid whenever $|P C(PQ)| < 1$. ◀

Using Proposition 5.11, $\mathcal{L}(W_U)(w)\mathcal{L}(S)(w) = \frac{15}{2}c \left(\frac{1}{4}\right)^{|w|_a} \left(\frac{1}{16}\right)^{|w|_b}$ for every w with $d(w) > 0$, so

$$\text{weight}(G^* \odot S) = \frac{15}{2}c \sum_{|w| \text{ odd}, d(w) > 0} \binom{n_a + n_b}{n_a} \left(\frac{1}{4}\right)^{n_a} \left(\frac{1}{16}\right)^{n_b}.$$

Writing $G(P, Q)$ for the sum in Lemma 5.14 and isolating the odd-length terms via $\frac{1-(-1)^n}{2}$,

$$\sum_{|w| \text{ odd}, d(w) > 0} \binom{n_a + n_b}{n_a} \left(\frac{1}{4}\right)^{n_a} \left(\frac{1}{16}\right)^{n_b} = \frac{G\left(\frac{1}{4}, \frac{1}{16}\right) - G\left(-\frac{1}{4}, -\frac{1}{16}\right)}{2}.$$

In both cases $PQ = \frac{1}{64}$, so $C(PQ) = 32 - 8\sqrt{15}$ and $\sqrt{1 - 4PQ} = \frac{\sqrt{15}}{4}$. Substituting $P = \frac{1}{4}$ gives $G\left(\frac{1}{4}, \frac{1}{16}\right) = \frac{120 - 16\sqrt{15}}{165}$, and $P = -\frac{1}{4}$ gives $G\left(-\frac{1}{4}, -\frac{1}{16}\right) = \frac{40 - 16\sqrt{15}}{105}$. Hence

$$\sum_{|w| \text{ odd}, d(w) > 0} \binom{n_a + n_b}{n_a} \left(\frac{1}{4}\right)^{n_a} \left(\frac{1}{16}\right)^{n_b} = \frac{8(25 + 4\sqrt{15})}{1155} =: \Sigma \approx 0.2805.$$

Setting $\text{weight}(G^* \odot S) = \frac{15}{2}c \Sigma$ equal to 1 and solving for c gives the following.

► **Proposition 5.15.** With

$$c = \frac{25 - 4\sqrt{15}}{20} \approx 0.4754,$$

we have $\text{weight}(G^* \odot S) = 1$, and by Proposition 5.13 this is the supremum of $\text{weight}(J \odot S)$ over all admissible J .

We fix S to use this value of c for the remainder of this section. Note that $c < \frac{1}{2}$, so in fact $\mathcal{L}(S)(w) < 1$ even for the best single word $w = a$ (where $d(w) = 1$): the threshold 1 is reached only in aggregate, by combining the (unboundedly long) tail of ever more numerous, individually low-scoring words with $d(w)$ close to 0, together with the comparatively rare but high-scoring words of large excess.

We now show that G^* cannot be the semantics of any WFA. Two facts are needed: that supports of non-negative WFAs are always regular, and that the support of G^* is not.

► **Proposition 5.16.** Let $\mathcal{W}$ be a non-negative WFA. Then $\text{supp}(\mathcal{L}(\mathcal{W})) := \{w : \mathcal{L}(\mathcal{W})(w) \neq 0\}$ is a regular language.

Proof. Since $\mathcal{W}$ is non-negative, $\mathcal{L}(\mathcal{W})(w) \neq 0$ if and only if at least one path from an initial state (positive α -entry) to a final state (positive β -entry), labelled by w , has positive weight along every transition used; no cancellation can occur among non-negative summands. This is exactly the condition recognised by the ordinary (Boolean) NFA obtained from $\mathcal{W}$ by keeping a state initial (respectively final) iff its α -entry (respectively β -entry) is positive, and keeping a transition iff its weight is positive. This NFA is finite, so its language, which equals $\text{supp}(\mathcal{L}(\mathcal{W}))$, is regular. ◀

► **Lemma 5.17.** $L^* := \{w \in \Sigma^* : |w| \text{ odd}, |w|_a > |w|_b\}$ is not regular.

Proof. We exhibit infinitely many pairwise Myhill–Nerode inequivalent words. Fix any $n < m$ of the same parity, and consider the suffix $z = b^{n+1}$. Then $a^n z = a^n b^{n+1}$ has odd length $2n + 1$ and $|a^n z|_a = n < n + 1 = |a^n z|_b$, so $a^n z \notin L^*$. On the other hand $a^m z = a^m b^{n+1}$ has odd length $m + n + 1$ (odd, since $m \equiv n \pmod{2}$) and $|a^m z|_a = m \geq n + 2 > n + 1 = |a^m z|_b$, so $a^m z \in L^*$. Hence z distinguishes a^n from a^m for every pair $n < m$ of equal parity, giving infinitely many pairwise inequivalent residues of L^* . By the Myhill–Nerode theorem, L^* is not regular. ◀

Since $\text{supp}(G^*) = L^*$, Proposition 5.16 and Lemma 5.17 immediately give the following.

► **Theorem 5.18.** G^* is not the semantics of any WFA.

Combining Proposition 5.13 and Theorem 5.18: G^* is the *unique* non-negative weighted language that is admissible for the residual problem ($0 \leq J \leq \mathcal{L}(W_U)$, $\text{weight}(J) = 1$) and achieves $\text{weight}(J \odot S) \geq 1$; every other admissible J satisfies $\text{weight}(J \odot S) < 1$ strictly. Since G^* has no WFA representative, no WFA I , let alone a stochastic one, can satisfy $\text{weight}(I \odot S) \geq 1$ while respecting $0 \leq \mathcal{L}(I) \leq \mathcal{L}(W_U)$.

► **Corollary 5.19.** The WCI instance (W_L, W_U, S) has no solution.

Note that S is not finitely insufficient (Definition 5.7): for every threshold V , infinitely many words have arbitrarily negative excess $d(w)$ and hence score below V . This instance therefore lies squarely outside the tractable fragment of Section 5.2.5, and the failure of that fragment’s hypothesis is not incidental, it is exactly what allows the optimum to require infinitely many excess levels simultaneously, forcing a non-regular support.

Despite Corollary 5.19, solutions can be approximated arbitrarily

well by actual WFAs. For $N \geq 1$, let

$$G_N := \sum_{\substack{|w| \leq N, |w| \text{ odd} \\ d(w) > 0}} \mathcal{L}(W_U)(w)[w] + \varepsilon_N[b],$$

$$\varepsilon_N := 1 - \sum_{\substack{|w| \leq N, |w| \text{ odd} \\ d(w) > 0}} \mathcal{L}(W_U)(w),$$

a finite sum of atomic automata (Definition 2.15). Since $\varepsilon_N \leq \varepsilon_1 = \frac{1}{16} < \mathcal{L}(W_U)(b) = \frac{15}{16}$, we have $0 \leq G_N \leq \mathcal{L}(W_U)$ pointwise, and $\text{weight}(G_N) = 1$ by construction, so by Theorem 4.9 G_N is (equivalent to) a stochastic WFA satisfying the hard constraints. By Lemma 5.12 the partial sums increase to 1 as $N \rightarrow \infty$, so $\varepsilon_N \rightarrow 0$, and since $\mathcal{L}(S)(b) = c/2$ is fixed,

$$\text{weight}(G_N \odot S) \longrightarrow \text{weight}(G^* \odot S) = 1 \quad (N \rightarrow \infty),$$

while $\text{weight}(G_N \odot S) < 1$ for every finite N , by uniqueness of the maximiser G^* (Proposition 5.13). Each G_N is thus a genuine stochastic WFA, satisfying the hard bounds exactly and missing the soft constraint by an amount that can be made as small as desired by choosing N large, but never zero.

This example refines the undecidability result of Theorem 5.4 in an important way: it shows that a WCI instance can fail to be solvable not because of some pathological interaction between ambiguity and cancellation, but purely because the *optimal* candidate distribution, unique, and forced by a tight budget constraint (Lemma 5.12), has a support that is fundamentally unrepresentable by a finite-state device with non-negative weights (Proposition 5.16). Every automaton involved is two states, deterministic, and hence trivially unambiguous, so the decidable containment result of Theorem 4.14 applies to *verifying* any given candidate against $\langle S, W_U \rangle$: had we been handed a specific stochastic WFA and asked whether it respects the hard bounds, the answer would have been immediate. What fails here is not containment-checking but the very existence of a WFA to check in the first place, a distinction that is easy to overlook, since decidability of verification is often taken, informally, as a proxy for the tractability of synthesis. This instance shows the two can come apart entirely, even within the most restricted, most well-behaved class of WFAs studied in this thesis.

This also clarifies the scope of Section 5.2.5. Finite insufficiency of S was used there to guarantee that only finitely many words need to be considered explicitly, with the (regular, since infinite but uniformly bounded-below in score) remainder handled in bulk. The present instance shows what can go wrong without that assumption: the soft constraint here rewards unboundedly long words increasingly steeply

as their letter-excess grows, so that no finite prefix of the ranking by score ever captures a fixed fraction of the optimum, and every word, however long, still leaves an infinite higher-scoring tail unaccounted for. Any general synthesis procedure for WCI must therefore either restrict attention to soft constraints excluding this behaviour (as in Section 5.2.5), settle for an approximate or limiting notion of solution (as the sequence $(G_N)_{N \geq 1}$ suggests), or accept that some instances, even startlingly simple, fully deterministic ones, admit no solution at all.

Taken together, the results of this chapter show that weighted control improvisation, despite being undecidable in full generality, is far from intractable in practice. Verification reduces cleanly to three checks that are each either exactly decidable in polynomial time or decidable up to an arbitrarily small tolerance, so a candidate improviser can always be validated with confidence. Synthesis is more delicate: the general problem is already undecidable for the simplest possible soft constraints, since undecidability follows already when the hard bounds are trivial. Nevertheless, the hard bounds themselves contribute no difficulty of their own: they can always be eliminated in favour of a single residual automaton and two numerical targets, and whenever the soft constraint is already satisfied by this reduction, the resulting hard-constraint fragment is always explicitly solvable. When the soft constraint must still be pursued, we have isolated the basic move that any synthesis procedure for it should be built around: whenever a word crosses the required average score, committing to it fully is a safe and lossless step. For the finitely insufficient soft constraints that arise naturally in examples such as Bob’s railway network, the park-routing scenario, and every WCI instance translated from classical control improvisation, this move can be iterated into a fully explicit, terminating synthesis algorithm with no oracle required. At the same time, Section 5.3 shows that this last restriction is not merely a matter of convenience: dropping it, even while keeping every automaton in the instance deterministic and two states large, can produce an instance whose unique optimal solution provably escapes the class of WFAs altogether. This progression, from a fully general but undecidable problem to a hierarchy of increasingly tractable fragments, punctuated by a sharp reminder of where that hierarchy runs out, mirrors the situation for classical control improvisation and for probabilistic automata more broadly, and suggests that the undecidability results of this chapter should be read not as an obstacle to using WCI, but as a guide to which specifications are worth designing in the first place.

6 Related Work

This thesis lies at the intersection of two research areas: weighted automata and probabilistic language models on the one hand, and control improvisation and randomised synthesis on the other. Although these fields developed largely independently, both address the problem of representing and generating structured behaviour subject to formal constraints. Weighted control improvisation combines ideas from both directions by using weighted automata as a specification formalism for synthesising probability distributions over words.

6.1 Weighted automata and rational series

Weighted automata generalise classical finite automata by associating a numerical value, rather than a Boolean acceptance verdict, with each word. The field originates with Schützenberger’s work on formal power series [22], and has since been developed into a general framework over arbitrary semirings; see the *Handbook of Weighted Automata* [7], Sakarovitch [21], and Mohri’s survey [18] for comprehensive treatments.

Particularly relevant here is the representation of weighted automata over the reals via products of transition matrices, which turns automata-theoretic questions into linear algebraic ones and underlies modern approaches to minimisation, characterised through forward and backward spaces [15]. The literature also studies containment and equivalence; while equivalence is decidable for many classes, containment is significantly harder, and recent work has clarified the decidability landscape in probabilistic and weighted settings [3, 4]. These containment-style questions are directly relevant to both verification and synthesis for weighted control improvisation.

A distinctive feature of this thesis is its emphasis on total language weight: whether the weights of all words in a weighted language sum to a finite value. This question is less prominent in classical weighted automata theory, which typically focuses on individual word weights, but it is essential here since probability distributions require finite normalisation constants. Our analysis of convergence relies on spectral methods, viewing a weighted automaton as a linear dynamical system

and reducing convergence to matrix stability, drawing on the broader literature on spectral radii and joint spectral radius theory [13, 14].

6.2 Probabilistic automata and stochastic languages

Probabilistic automata, introduced by Rabin [20], extend finite automata with probabilistically chosen transitions, associating a probability with each word rather than a classical accept/reject verdict. Paz’s monograph [19] remains a standard reference, and the connection to weighted automata is immediate: probabilistic automata are weighted automata whose weights satisfy probabilistic constraints. Related work on stochastic languages, that is, finite-state-representable probability distributions over words, includes realisation problems studied by Carlyle and Paz [2] and rational stochastic languages studied by Denis and Esposito [5]; a weighted control improviser is precisely such a stochastic generator.

Probabilistic automata are also notorious for difficult algorithmic properties, with many decision problems becoming undecidable once probabilities are introduced, though some subclasses retain decidable verification [10, 4]. Since a valid WCI improviser must itself define a probability distribution, several undecidability results in this thesis can be viewed as instances of these well-known hardness phenomena.

6.3 Control improvisation

Control improvisation, introduced by Fremont, Donzé, Seshia, and Wessel [8], addresses the synthesis of randomised behaviours satisfying hard and soft constraints. The motivating observation is that deterministic behaviour, even when correct, is often undesirable: a fixed patrol route can be anticipated, a fixed test path defeats its purpose, and a fixed melody becomes repetitive. Classical control improvisation formalises this through a hard constraint, a soft constraint, lower and upper probability bounds, and a finite length bound, asking not merely for a feasible execution but for a probability distribution over feasible executions. The original work gave efficient synthesis procedures for automata-based specifications while showing more expressive variants to be computationally hard or undecidable [8].

6.4 Extensions of control improvisation

Control improvisation has since been extended in several directions. Akkaya et al. combined it with probabilistic temporal specifications [1], connecting it to verification techniques from cyber-physical systems. Fremont and Seshia introduced reactive control improvisation, in which behaviour evolves through interaction with an environment [9], moving beyond finite-word generation towards reactive synthesis. Vazquez-Chanlatte et al. proposed entropy-guided control improvisation, optimising entropy directly rather than imposing randomness through probability bounds [24], connecting the framework to information theory and stochastic games. Together, these extensions suggest that control improvisation is best understood as a general paradigm for synthesising constrained random behaviour, rather than a single fixed formalism.

6.5 Positioning of WCI

Weighted control improvisation extends this paradigm by replacing Boolean language specifications with weighted ones. Classical CI treats every admissible word qualitatively, with soft constraints offering only a partial, still fundamentally Boolean, remedy. By specifying hard and soft constraints as weighted automata, WCI expresses word-dependent probability bounds and genuine quantitative objectives within a single algebraic framework. From the perspective of weighted automata theory, the contribution is complementary: where that literature has focused on representation, analysis, and verification, this thesis studies synthesis, constructing a weighted language satisfying quantitative constraints rather than merely analysing one.

This combination is largely unexplored: weighted automata have been used extensively to represent quantitative systems, and control improvisation extensively to synthesise constrained randomness, but synthesis directly over weighted language specifications has received comparatively little attention. The framework developed in this thesis is intended as a step towards closing that gap.

7 Conclusion

This thesis has developed weighted control improvisation (WCI) as a framework for synthesising probabilistic generators under quantitative constraints. The central idea was to replace the Boolean specification language of classical control improvisation with weighted finite automata, so that hard constraints, probability bounds, and soft objectives can all be expressed in a uniform linear algebraic setting. This makes it possible to model not only which words are admissible, but also how desirable they are, how much probability mass they should receive, and how these requirements should be balanced over potentially unbounded behaviours.

We studied the theory of weighted languages and weighted finite automata as the mathematical foundation of WCI. In particular, we studied finite weight, the matrix-based semantics of WFAs, closure under natural operations, and minimisation via forward and backward spaces.

A main contribution was the formal definition of WCI itself. Classical CI is recovered as a special case by encoding the Boolean hard and soft languages as weighted automata and by using uniform probability bounds. At the same time, the weighted setting is strictly more expressive: it allows word-dependent lower and upper bounds, arbitrary real-valued scoring functions, and improvisations of unbounded length. This extra expressiveness is essential for applications in which desirability is inherently quantitative rather than merely Boolean.

The thesis also analysed the computational status of the resulting synthesis and verification problems. In full generality, both problems are undecidable, which is consistent with the difficulty of related problems for probabilistic and weighted automata. Nevertheless, the thesis identifies useful tractable fragments. Total weight can be computed efficiently when it is finite, the containment problem admits an approximate decision procedure, and for non-negative instances a greedy synthesis algorithm exists. This gives a clear picture of the trade-off that lies at the heart of the framework: richer specifications are more expressive, but they also require more careful algorithmic treatment.

The examples in the thesis, including the railway and park-routing scenarios, illustrate why this trade-off is worthwhile. In both cases, the

generator must be randomised, yet not arbitrary; it must satisfy hard feasibility requirements, distribute probability in a controlled way, and optimise a quantitative objective. WCI captures these requirements directly, and the examples show that the weighted formalism is well suited to applications in which diversity, robustness, and expected quality all matter at once.

The main limitation of the present work is that the most general synthesis and verification problems remain undecidable. In practice, this means that the framework is best suited to settings where the specification has additional structure, or where approximation is acceptable. Another limitation is that useful lower- and upper-bound automata are often close in structure to the desired solution, so constructing good specifications may require substantial domain knowledge. These are not defects of the model itself, but they do shape how it can be applied effectively.

7.1 Future work

There are several natural directions in which this work could be extended. The most immediate is to sharpen the decidability boundary established in Chapter 5. We have shown that synthesis is undecidable in full generality, yet decidable for finitely insufficient soft constraints, and similarly that containment is undecidable in general but decidable under unambiguity assumptions. These results leave a considerable gap between the tractable fragments we identify and the general undecidable problem, and it remains unclear exactly where the boundary lies.

A second direction is expressive power. Weighted control improvisation is formulated over weighted finite automata, but the same basic idea could potentially be lifted to richer specification formalisms, such as weighted transducers, probabilistic temporal logics, or reactive and game-theoretic settings. In particular, it would be interesting to combine the quantitative nature of WCI with the interaction-based structure of reactive control improvisation and stochastic games. This could lead to a framework for generating infinite behaviours under both environment-dependent and quantitative constraints. Such extensions would also shed light on the boundary question above, since richer automata models tend to come with their own, often better-understood, decidability landscapes that could be compared against the one developed here.

A third direction is implementation and approximation. Practical adoption would benefit from a full software toolchain for constructing WFAs, checking total weight, solving synthesis instances, and validating candidate improvisers. We have a basic prototype implementation that allows for the synthesis and verification of simple WCI instances [17], but we have not tested this on large inputs.

Overall, this thesis shows that weighted control improvisation provides a natural and expressive framework for quantitative generation problems. It generalises classical control improvisation, retains a substantial amount of algebraic structure, and opens the door to new synthesis problems over infinite or unbounded behaviours. At the same time, it raises several algorithmic and theoretical questions that remain open. These questions make WCI a promising basis for further research, both in theory and in applications where randomness must be controlled rather than eliminated.

Bibliography

- [1] Ilge Akkaya, Daniel J. Fremont, Rafael Valle, Alexandre Donzé, Edward A. Lee, and Sanjit A. Seshia. Control improvisation with probabilistic temporal specifications. *CoRR*, abs/1511.02279, 2015.
- [2] J. W. Carlyle and A. Paz. Realizations by stochastic finite automata. *Journal of Computer and System Sciences*, 5(1):26–40, 1971.
- [3] Laure Daviaud. Containment and equivalence of weighted automata: Probabilistic and max-plus cases. In *Language and Automata Theory and Applications (LATA 2020)*, volume 12038 of *Lecture Notes in Computer Science*, pages 17–32. Springer, 2020.
- [4] Laure Daviaud, Marcin Jurdziński, Ranko Lazić, Filip Mazowiecki, Guillermo A. Pérez, and James Worrell. When are emptiness and containment decidable for probabilistic automata? *Journal of Computer and System Sciences*, 119:78–96, 2021.
- [5] François Denis and Yann Esposito. Rational stochastic languages. *CoRR*, abs/cs/0602093, 2006.
- [6] Alexandre Donzé, Sophie Libkind, Sanjit A. Seshia, and David Wessel. Control improvisation with application to music. Technical Report UCB/EECS-2013-183, EECS Department, University of California, Berkeley, November 2013.
- [7] Manfred Droste, Werner Kuich, and Heiko Vogler, editors. *Handbook of Weighted Automata*. Springer, 2009.
- [8] Daniel J. Fremont, Alexandre Donzé, Sanjit A. Seshia, and David Wessel. Control improvisation. In *FSTTCS*, volume 45 of *LIPIcs*, pages 463–474. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2015.
- [9] Daniel J. Fremont and Sanjit A. Seshia. Reactive control improvisation. In *CAV (1)*, volume 10981 of *Lecture Notes in Computer Science*, pages 307–326. Springer, 2018.

- [10] Hugo Gimbert and Youssef Oualhadj. Probabilistic automata on finite words: Decidable and undecidable problems. In *Automata, Languages and Programming (ICALP 2010), Part II*, volume 6199 of *Lecture Notes in Computer Science*, pages 527–538. Springer, 2010.
- [11] Kosaburo Hashiguchi, Kenichi Ishiguro, and Shuji Jimbo. Decidability of the equivalence problem for finitely ambiguous finite automata. *International Journal of Algebra and Computation*, 12(3):445–461, 2002.
- [12] Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, Cambridge, UK, 1991.
- [13] Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, 2 edition, 2013.
- [14] Raphaël Jungers. *The Joint Spectral Radius: Theory and Applications*. Springer Science & Business Media, 2009.
- [15] Stefan Kiefer. Notes on equivalence and minimization of weighted automata. *CoRR*, abs/2009.01217, 2020.
- [16] Daniel Krob. The equality problem for rational series with multiplicities in the tropical semiring is undecidable. *International Journal of Algebra and Computation*, 4(3):405–425, 1994.
- [17] Jasper Laumen. Control improvisation. https://github.com/JLaumen/control_improvisation, 2026.
- [18] Mehryar Mohri. *Weighted Automata Algorithms*, pages 213–254. Springer Berlin Heidelberg, Berlin, Heidelberg, 2009.
- [19] Azaria Paz. *Introduction to Probabilistic Automata*. Academic Press, New York, 1971.
- [20] Michael O. Rabin. Probabilistic automata. *Information and Control*, 6(3):230–245, 1963.
- [21] Jacques Sakarovitch. *Elements of Automata Theory*. Cambridge University Press, 2009.
- [22] Marcel-Paul Schützenberger. On the definition of a family of automata. *Information and Control*, 4(2–3):245–270, 1961.
- [23] Milind Tambe. *Security and Game Theory: Algorithms, Deployed Systems, Lessons Learned*. Cambridge University Press, 2011.
- [24] Marcell Vazquez-Chanlatte, Sebastian Junges, Daniel J. Fremont, and Sanjit A. Seshia. Entropy-guided control improvisation. In *Robotics: Science and Systems*, 2021.

- [25] Zhengyu Yin, Albert Xin Jiang, Matthew P. Johnson, Milind Tambe, Christopher Kiekintveld, Kevin Leyton-Brown, Tuomas Sandholm, and John P. Sullivan. TRUSTS: Scheduling randomized patrols for fare inspection in transit systems using game theory. *AI Magazine*, 33(4):59–72, 2012.